Unidad 1. Regresión lineal simple y correlación

Medidas de dispersión

Suma de los cuadrados de las desviaciones de los valores de X con respecto a su media:

$$S_{xx} = \sum_{i=1}^{n} (X_i - \bar{X})^2$$

Suma de los productos de las desviaciones de los valores de X y Y con respecto a sus medias:

$$S_{xy} = \sum_{i=1}^{n} (X_i - \bar{X})(Y_i - \bar{Y})$$

Suma de los cuadrados de las desviaciones de los valores de Y con respecto a su media:

$$S_{xx} = \sum_{i=1}^{n} (Y_i - \bar{Y})^2$$

Coeficiente de correlación y de determinación

Coeficiente de correlación de Pearson:

$$r = \frac{S_{xy}}{\sqrt{S_{xx} \cdot S_{yy}}}$$

Coeficiente de determinación:

$$r^2$$

Recta de regresión ajustada

La regresión lineal ajustada se representa mediante estadísticos:

$$\hat{Y} = b_0 + b_1 X$$

donde $\hat{Y}$ representa el valor de Y obtenido mediante la recta de regresión ajustada (no la verdadera Y). Los estadísticos b_0 y b_1 se calculan de la siguiente manera:

$$b_1 = \frac{S_{xy}}{S_{xx}}$$
$$b_0 = \bar{Y} - b_1 \bar{X}$$

Cálculo de residuales

Residuales:

$$e_i = Y_i - \hat{Y}_i$$

Sumas de cuadrados SS (Sum of Squares)

Suma de los Cuadrados de los Errores:

$$SSE = \sum_{i=1}^{n} (Y_i - \hat{Y}_i)^2 = \sum_{i=1}^{n} e_i^2$$

Suma total de cuadrados:

$$SST = \sum_{i=1}^{n} (Y_i - \bar{Y})^2$$

Suma de cuadrados de regresión:

$$SSR = \sum_{i=1}^{n} (\hat{Y}_i - \bar{Y})^2$$

- SST: Mide la variabilidad total de los datos observados.
- SSR: Mide la variabilidad de los datos que el modelo de regresión explica.
- SSE: Mide la variabilidad no explicada por el modelo (es decir, los residuos).

Intervalo de confianza

Estadístico de prueba t:

$$t = \frac{b_1}{SE(b_1)}$$

Error estándar de b_1:

$$SE(b_1) = \frac{\sqrt{SSE/(n-2)}}{\sqrt{S_{xx}}}$$

o

$$SE(b_1) = \frac{\sqrt{\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2/(n-2)}}{\sqrt{S_{xx}}}$$

Intervalo de confianza para b_1:

$$b_1 - t_{\alpha/2} \cdot SE(b_1) < \beta_1 < b_1 + t_{\alpha/2} \cdot SE(b_1)$$

donde n representa la cantidad de pares de datos.

Comprobación de supuestos

Comprobar suposiciones:

- Test de shapiro a los residuales e_i: Para comprobar si la distribución es normal sobre la recta.
- Grafico X vs Y: Para observar si los datos soportan la suposición de linealidad.
- Gráfico de residuales: Para observar si los datos soportan la suposición de linealidad, complementario al coeficiente de correlación
- Test de Breusch-Pagan: Para detectar heteroscedasticidad en regresion lineal

Test de Shapiro:
`from scipy.stats import shapiro`
Después, se obtiene el valor-p:
`_, valor_p_sh = shapiro(data)`

- H_0: Los datos siguen una distribución normal
- H_1: Los datos no siguen una distribución normal

Test de Breusch-Pagan:
`from statsmodels.stats.api import het_breuschpagan`
Después, se obtiene el valor-p:
`_, valor_p_bp, _, _ = het_breuschpagan(residuales, X)`

- H_0: Hay homoscedasticidad
- H_1: Hay heteroscedasticidad

ANOVA en regresión lineal

Fuente de variación	Suma de cuadrados (SS)	Grados de libertad (df)	Promedio de los cuadrados (MS)	Estadístico F
Regresión	SSR	p	$MSR = \frac{SSR}{p}$	$F = \frac{MSR}{MSE}$
Error	SSE	$n-p-1$	$MSE = \frac{SSE}{n-p-1}$	
Total	SST	$n-1$		

donde p es el número de parámetros para la recta de regresión ajustada (en la regresión simple $p = 1$). Las hipótesis son:

$$H_0 : \beta_1 = 0$$
$$H_1 : \beta_1 \neq 0$$

Problemario de la Unidad 1

Problema 1

Un profesor intenta mostrar a sus estudiantes la importancia de los exámenes cortos, aun cuando el 90% de la calificación final esté determinada por los exámenes parciales. Él cree que cuanto más altas sean las calificaciones de los exámenes cortos, más alta será la calificación final. Seleccionó una muestra aleatoria de 15 estudiantes de su clase con los siguientes datos:

Promedio de exámenes cortos	Promedio final
59	64
92	84
72	77
90	80
95	77
87	81
89	80
77	84
76	80
65	69
97	83
42	40
94	78
62	65
91	90

1. Establesca una variable dependiente (Y) y una variable independiente (X).
2. Realice un diagrama de dispersión para estos datos.
3. ¿Los datos soportan la suposición de linealidad?
4. Calcule el coeficiente de correlación e interprete el resultado.
5. Calcule el coeficiente de determinación e interprete el resultado.
6. Obtenga la recta de regresión ajustada y grafíquelo sobre el gráfico de dispersión.
7. Obtenga un intervalo de confianza del 95% para la pendiente de la recta de regresión ajustada (b_1)
8. Calcule los residuales y trace un nuevo gráfico de dispersión. Comente, ¿Parece que se verifican los supuestos?
9. Realice la prueba de Shapiro para los residuales y comente el resultado.
10. Realice la prueba de Brausch-Pagan para los residuales y comente el resultado.
11. Tres estudiantes sacaron 70, 75 y 84 de calificación. Según la recta de regresión ajustada, ¿cuáles son los resultados esperados para estos tres alumnos?
12. Realice una tabla ANOVA e interprete el resultado.

```
import pandas as pd
df=pd.DataFrame({
    "promedio_de_examenes_cortos":[59,92,72,90,95,87,89,77,76,65,97,42,94,62,91],
    "promedio_final":[64,84,77,80,77,81,80,84,80,69,83,40,78,65,90]
})
df
```

	promedio_de_examenes_cortos	promedio_final
0	59	64
1	92	84
2	72	77
3	90	80
4	95	77
5	87	81
6	89	80
7	77	84
8	76	80
9	65	69
10	97	83
11	42	40
12	94	78
13	62	65
14	91	90

Próximos pasos: Generar código con df New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 2 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   promedio_de_examenes_cortos  15 non-null     int64
 1   promedio_final               15 non-null     int64
dtypes: int64(2)
memory usage: 372.0 bytes
```

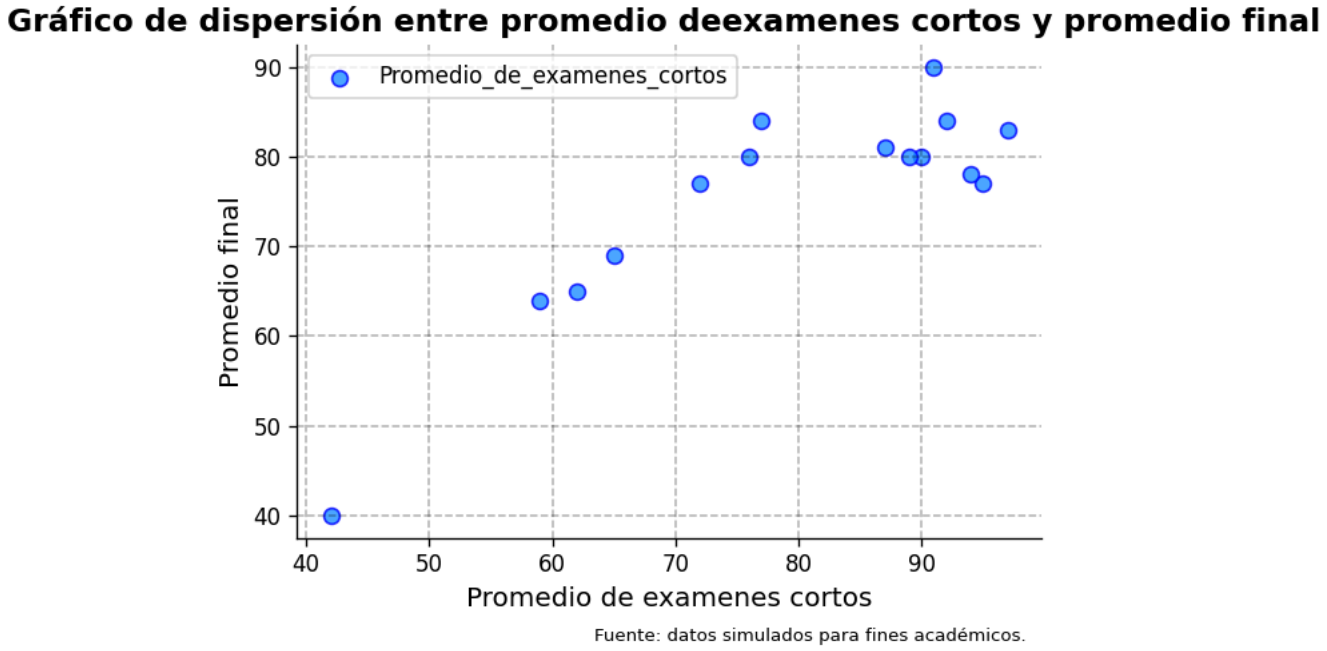
```
df.describe()
```

	promedio_de_examenes_cortos	promedio_final
count	15.000000	15.000000
mean	79.200000	75.466667
std	16.297239	12.123570
min	42.000000	40.000000
25%	68.500000	73.000000
50%	87.000000	80.000000
75%	91.500000	82.000000
max	97.000000	90.000000

```
#1.Establece una variable dependiente(y) y una variable independiente (x)
y=df["promedio_final"]
x=df["promedio_de_examenes_cortos"]
```

```
# @title
# 2. Realice un diagrama de dispersión para estos datos.
import matplotlib.pyplot as plt

# --- Configuración general del gráfico ---
plt.figure(
    figsize=(6, 4),   # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120           # resolución del gráfico
)

# --- Gráfico de dispersión ---
plt.scatter(
    x, y,
    marker="o",       # forma
    color='dodgerblue',     # color de los puntos
    edgecolor='blue',    # borde de los puntos
    alpha=0.8,            # transparencia
    s=50,                 # tamaño de los puntos
    label='Promedio_de_examenes_cortos' # etiqueta para la leyenda
)

# --- Gráfico de línea ---
# plt.plot(
#     x, y,
#     color='orange',   # color de la línea
#     linewidth=2.2,        # grosor de la línea
#     linestyle='-',        # estilo de línea
#     marker='o',           # marcador en cada punto
#     markersize=5,         # tamaño del marcador
#     markerfacecolor='white',
#     markeredgecolor='black',
#     label='Datos observados'
# )

# --- Título ---
plt.title(
    'Gráfico de dispersión entre promedio deexamenes cortos y promedio final ',
    fontsize=14,
    fontweight='bold'
)

# --- Etiquetas de los ejes ---
plt.xlabel(
    'Promedio de examenes cortos',
    fontsize=12
)
```

```
plt.ylabel(
    'Promedio final',
    fontsize=12
)

# --- Fuente de los ticks ---
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# --- Márgenes ---
plt.margins(x=0.05, y=0.05)  # espacio extra alrededor de los datos
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Para eliminar márgenes completamente, usar:
# plt.margins(0)

# --- Cuadrícula (opcional, pero didáctica) ---
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.9,
    alpha=0.3,
    color="black"
)

# --- Leyenda ---
plt.legend(
    fontsize=10,
    loc='best',
    frameon=True
)

# --- Nota al pie ---
plt.text(
    0.4, -0.2,
    'Fuente: datos simulados para fines académicos.',
    fontsize=8,
    ha='left',
    va='center',
    transform=plt.gca().transAxes
)

# --- Guardar gráfico ---
plt.savefig(
    "grafico de dispersión",
    bbox_inches='tight'
    )

plt.show()
```



```
#4 calcula el coeficiente de correcion e interpreta el resultado
from scipy.stats import pearsonr
r, valor_p= pearsonr(x,y)#p prueba de hipotesis
#corelacion: el cambio en una variable esta vinculado con el cambio en otra variable
print(f"Coeficiente de correlacion(r):{r:0.4f}")
print(f"valor-p:{valor_p:0.4f}")
#Ho:si hay correlacion
```

```
Coeficiente de correlacion(r):0.8646
valor-p:0.0000
```

```
#5 calcule el coeficiente de determinacion
print(f"Coeficiente de determinacion:{r**2:0.4f}")#dos asterisco elebando la potencia
#un asterisco es multiplicacion
```

```
Coeficiente de determinacion:0.7475
```

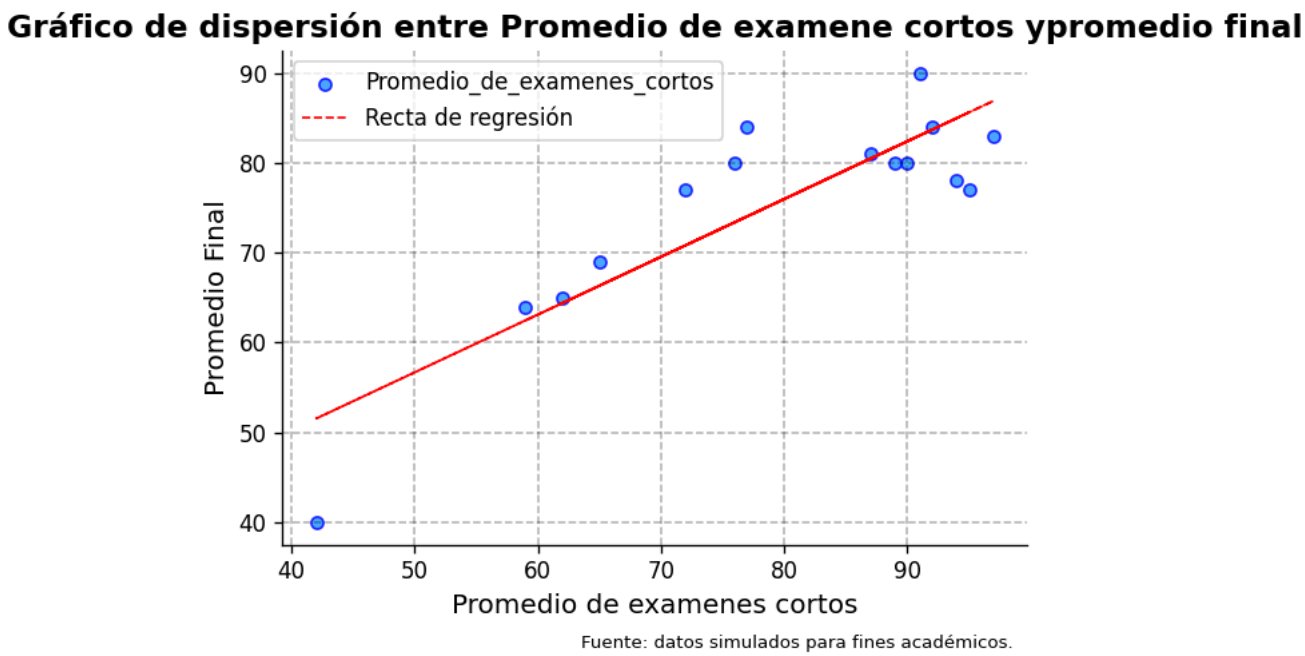
```
# 6. Obtenga la recta de regresión ajustada y grafíquelo sobre el
# gráfico de dispersión

import statsmodels.api as sm
x_constante = sm.add_constant(x)
modelo = sm.OLS(y, x_constante).fit()
y_calculada = modelo.predict(x_constante)

modelo.params
```

	0
const	24.526822
promedio_de_examenes_cortos	0.643180

dtype: float64

```
# @title
# 2. Realice un diagrama de dispersión para estos datos.
import matplotlib.pyplot as plt

# --- Configuración general del gráfico ---
plt.figure(
    figsize=(6, 4),   # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120           # resolución del gráfico
)

# --- Gráfico de dispersión ---
plt.scatter(
    x, y,
    marker="o",       # forma
    color='dodgerblue',     # color de los puntos
    edgecolor='blue',    # borde de los puntos
    alpha=0.8,            # transparencia
    s=30,                 # tamaño de los puntos
    label='Promedio_de_examenes_cortos' # etiqueta para la leyenda
)

# --- Gráfico de línea ---
plt.plot(
    x, y_calculada,
    color='red',   # color de la línea
    linewidth=1.0,        # grosor de la línea
    linestyle='--',        # estilo de línea
    marker='o',           # marcador en cada punto
    markersize=0,         # tamaño del marcador
    markerfacecolor='white',
    markeredgecolor='black',
    label='Recta de regresión'
)

# --- Título ---
plt.title(
    'Gráfico de dispersión entre Promedio de examene cortos ypromedio final',
    fontsize=14,
    fontweight='bold'
)

# --- Etiquetas de los ejes ---
plt.xlabel(
```

```
    'Promedio de examenes cortos',
    fontsize=12
)

plt.ylabel(
    'Promedio Final',
    fontsize=12
)

# --- Fuente de los ticks ---
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# --- Márgenes ---
plt.margins(x=0.05, y=0.05)  # espacio extra alrededor de los datos
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Para eliminar márgenes completamente, usar:
# plt.margins(0)

# --- Cuadrícula (opcional, pero didáctica) ---
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.9,
    alpha=0.3,
    color="black"
)

# --- Leyenda ---
plt.legend(
    fontsize=10,
    loc='best',
    frameon=True
)

# --- Nota al pie ---
plt.text(
    0.4, -0.2,
    'Fuente: datos simulados para fines académicos.',
    fontsize=8,
    ha='left',
    va='center',
    transform=plt.gca().transAxes
)

# --- Guardar gráfico ---
plt.savefig(
    "grafico de dispersión",
    bbox_inches='tight'
    )

plt.show()
```




```
modelo.params
```

	0
const	24.526822
promedio_de_examenes_cortos	0.643180

dtype: float64

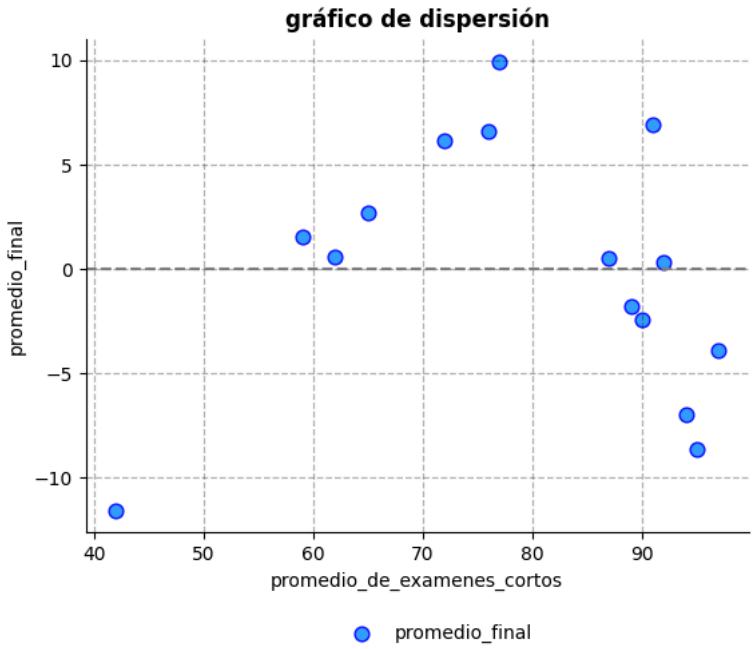
```
# 7. Obtenga un intervalo de confianza del 95% para la pendiente de la recta de
# regresión ajustada ( b1 )

# intervalo de confianza = 1 - alpha

modelo.conf_int(alpha = 0.05)
```

	0	1
const	6.441997	42.611646
promedio_de_examenes_cortos	0.419219	0.867141

```
#8 Residuales
residuales=modelo.resid
#Gráfico de dispersión
plt.scatter(
    x,residuales, #variables a graficar
    marker="o",
    color="dodgerblue",
    edgecolor="blue", #color del borde
    alpha=0.9, #transparencia
    s=60, #tamaño de cada punto
    label="promedio_final" #etiqueta
)
#Título
plt.title(
    "gráfico de dispersión",
    fontsize=12, #tamaño de fuente
    fontweight="bold" #bold:negritas
)
#Etiqueta eje x
plt.xlabel(
    "promedio_de_examenes_cortos",
    fontsize=10
)
#Etiqueta eje y
plt.ylabel(
    "promedio_final",
    fontsize=10
```

```
)
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)
#márgenes
plt.margins(x=0.05,y=0.05)
plt.gca().spines[["top","right"]].set_visible(False)
#Malla
plt.grid(
    visible=True,
    linestyle="--",
    linewidth=0.9, #ancho de línea
    alpha=0.3, #transparencia
    color="black"

)
#Leyenda
plt.legend(
    fontsize=10,
    loc="best",
    frameon=False,
    bbox_to_anchor=(0.7,-0.15)
)
#Guardar imagen
plt.savefig(
    "dispersión",
    bbox_inches="tight"
)
plt.axhline(y=0, color="gray",linestyle="--") #lineas horizontal
```

```
<matplotlib.lines.Line2D at 0x7f062c9fa8a0>
```



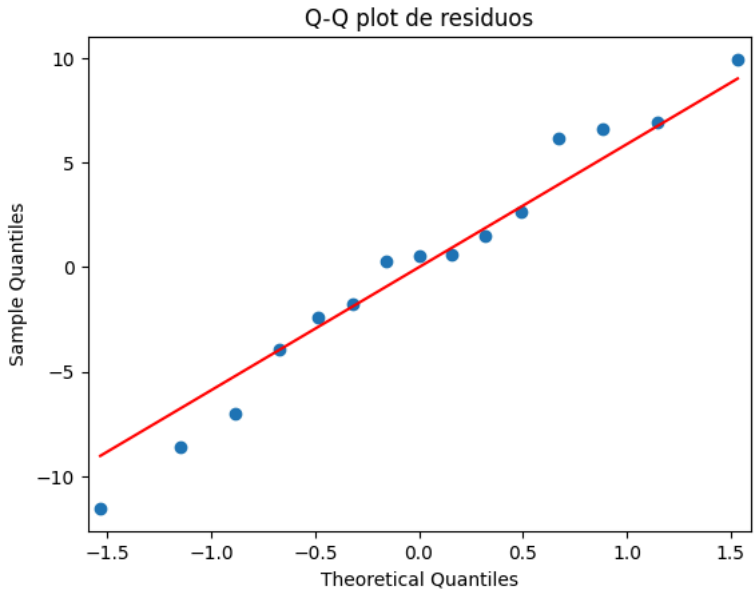

```
#9 Test de Shapiro
from scipy.stats import shapiro
estadistico,valor_p=shapiro(residuales)
print(f"valor_p:{valor_p:0.4f}")
```

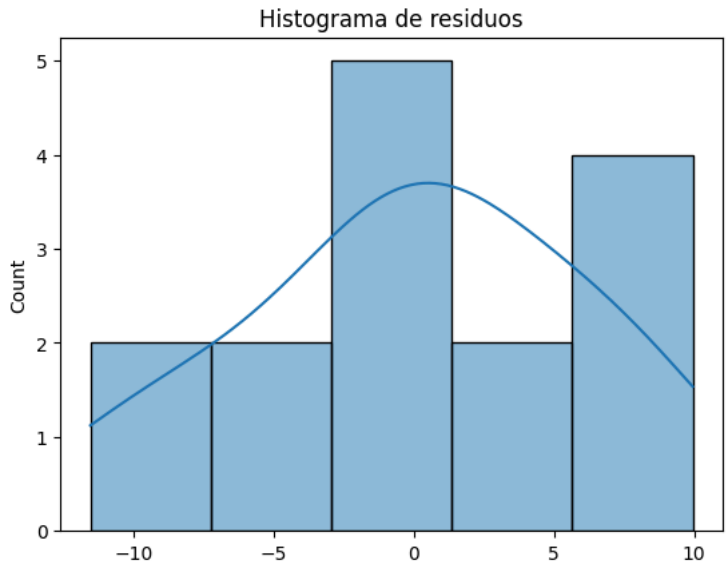
```
valor_p:0.9018
```

```
from scipy.stats import shapiro
import matplotlib.pyplot as plt
import seaborn as sns

# Prueba de Shapiro-Wilk
stat, valor_p_sh = shapiro(residuales)
print(f"valor-p (Shapiro) = {valor_p_sh}")

# Visualización: Q-Q plot
sm.qqplot(residuales, line='s')
plt.title("Q-Q plot de residuos")
plt.show()

# Histograma
sns.histplot(residuales, kde=True)
```

```
plt.title("Histograma de residuos")
plt.show()
```

```
valor-p (Shapiro) = 0.901827735700704
```






```
#10 Test de Breusch-Pagan
from statsmodels.stats.api import het_breuschpagan
estadistico_1,valor_p_1,estadistico_2,valor_p_2=het_breuschpagan(residuales,x_constante)
print(f"valor_p:{valor_p_1:0.4f}")
# si el valor es menor a sigma s rechaza H0
```

```
valor_p:0.2289
```

```
modelo.predict([1,70])
#x=75 y=72.77
#=84 y=78.55
```

```
array([69.54941193])
```

```
modelo.predict([1,75])
```

```
array([72.76531124])
```

```
modelo.predict([1,84])
```

```
array([78.55393001])
```

```
# 12. Realice una tabla ANOVA e interprete el resultado.

# test de ANOVA (Analysis of Variance)
```

```
# H0: beta_1 = 0   (No hay correlación)
# H1: beta_1 ≠ 0   (Sí hay correlación)

from statsmodels.formula.api import ols
# Y ~ X
modelo_lineal = ols('promedio_final ~promedio_de_examenes_cortos ', data = df).fit()
tabla_anova = sm.stats.anova_lm(modelo_lineal)
tabla_anova
```

	df	sum_sq	mean_sq	F	PR(>F)
promedio_de_examenes_cortos	1.0	1538.228959	1538.228959	38.492412	0.000032
Residual	13.0	519.504375	39.961875	NaN	NaN

Próximos pasos: Generar código con `tabla_anova` New interactive sheet

Problema 2

William Hawkins, vicepresidente de personal de la International Motors, trabaja en la relación entre el salario de un trabajador y el porcentaje de ausentismo. Hawkins dividió el intervalo de salarios de International en 12 grados o niveles (1 es el menor grado, 12 el más alto) y después muestreó aleatoriamente a un grupo de trabajadores. Determinó el grado de salario de cada trabajador y el número de días que ese empleado había faltado en los últimos 3 años.

Categoría de salario	**11**	**10**	**8**	**5**	**9**	**7**	**3**
Ausencias	18	17	29	36	11	28	35
Categoría de salario	**11**	**8**	**7**	**2**	**9**	**8**	**3**
Ausencias	14	20	32	39	16	31	40

1. Establesca una variable dependiente (Y) y una variable independiente (X).
2. Realice un diagrama de dispersión para estos datos.
3. ¿Los datos soportan la suposición de linealidad?
4. Calcule el coeficiente de correlación e interprete el resultado.
5. Calcule el coeficiente de determinación e interprete el resultado.
6. Obtenga la recta de regresión ajustada y grafíquelo sobre el gráfico de dispersión.
7. Obtenga un intervalo de confianza del 95% para la pendiente de la recta de regresión ajustada (b_1)
8. Calcule los residuales y trace un nuevo gráfico de dispersión. Comente, ¿Parece que se verifican los supuestos?
9. Realice la prueba de Shapiro para los residuales y comente el resultado.
10. Realice la prueba de Brausch-Pagan para los residuales y comente el resultado.
11. Utiliza la recta de regresión para interpolar dos valores y extrapolar uno. Comenta estos resultados.
12. Realice una tabla ANOVA e interprete el resultado.

```
import pandas as pd
df=pd.DataFrame({
    "categoria_salario":[11,10,8,5,9,7,3,11,8,7,2,9,8,3],
    "ausencias":[18,17,29,36,11,28,35,14,20,32,39,16,31,40]
    })
df
```

	categoria_salario	ausencias
0	11	18
1	10	17
2	8	29
3	5	36
4	9	11
5	7	28
6	3	35
7	11	14
8	8	20
9	7	32
10	2	39
11	9	16
12	8	31
13	3	40

Próximos pasos: Generar código con df New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14 entries, 0 to 13
Data columns (total 2 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   categoria_salario  14 non-null     int64
 1   ausencias          14 non-null     int64
dtypes: int64(2)
memory usage: 356.0 bytes
```

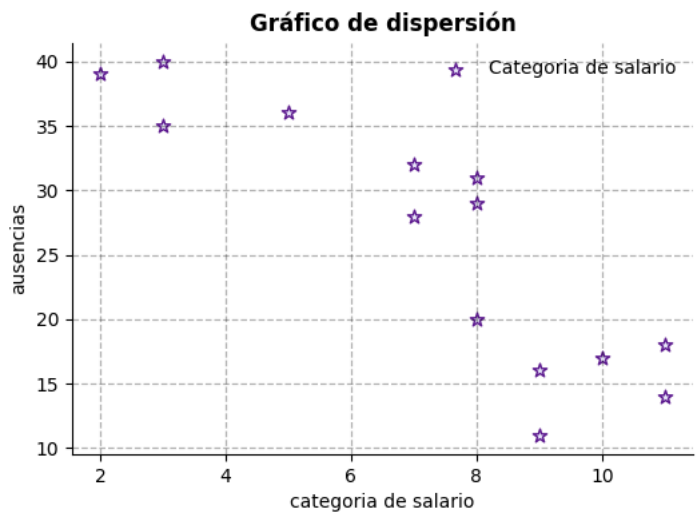
```
df.describe()
```

	categoria_salario	ausencias
count	14.000000	14.000000
mean	7.214286	26.142857
std	2.939874	9.882830
min	2.000000	11.000000
25%	5.500000	17.250000
50%	8.000000	28.500000
75%	9.000000	34.250000
max	11.000000	40.000000

```
#1 Definir variables
x=df["categoria_salario"]
y=df["ausencias"]
```

```
#2
from matplotlib.lines import lineStyles
import matplotlib.pyplot as plt
#configuración general
plt.figure(
    figsize=(6,4),  #tamaño (ancho, alto)
    dpi=100         #resolución
)
#Gráfico de dispersión
plt.scatter(
    x,y,
    marker="*",
    color="lavender",
    edgecolor="indigo", #color de borde
    alpha=0.9, #transparencia
    s=60, #tamaño de punto
```

```
    label="Categoria de salario" #etiqueta
)
#Título
plt.title(
    "Gráfico de dispersión",
    fontsize=12, #tamaño de fuente
    fontweight="bold" #bold:negritas
)
#Etiquetas eje x
plt.xlabel(
    "categoria de salario",
    fontsize=10,
)
#Etiqueta eje y
plt.ylabel(
    "ausencias",
    fontsize=10
)
#tamaño de los tricks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

#Márgenes
plt.margins(x=0.05,y=0.05)
plt.gca().spines[["top","right"]].set_visible(False)

plt.grid(
    visible=True,
    linestyle="--",
    linewidth=0.9, #ancho de line
    alpha=0.3, #transparencia
    color="black"
)
    #Leyenda
plt.legend(
        fontsize=10,
        loc="best",
        frameon=False
        #bbox_to_anchor=(0.5.-0.15)
)
        #Guardar imagen
plt.savefig(
            "Dispersión",
            bbox_inches="tight"
        )
```




```
# 3 P R E G U N T A : ¿Soporta la suposición de linealidad?
# SI, los datos muestran una tendencia aproximadamente lineal.
```

```
#4 Calcula el coeficiente de correlación e interpreta el resultado
from scipy.stats import pearsonr
r, valor_p=pearsonr(x,y)
print(f"coeficiente de correlación (r):{r:0.4f}")
print(f"valor_p:{valor_p:0.4f}")

# Test de Pearson
# H0: rho = 0     (No hay correlación)
```

```
# H1: rho ≠ 0     (Sí hay correlación)
# alpha = 0.05

#     CONCLUSION DE PEARSON

#       COMO EL VALOR DE r ES IGUAL -0.8801
#       SE DEDUCE QUE H1 ES VERDADEA, POR LO CUAL hay correlación
```

```
coeficiente de correlación (r):-0.8801
valor_p:0.0000
```

```
#5 Calcule el coeficiente de determinación 0= r*r
print(f"coeficiente de determinación:{r**2:0.4f}")
```

```
coeficiente de determinación:0.7746
```

```
modelo.params
```

	0
const	24.526822
promedio_de_examenes_cortos	0.643180

dtype: float64

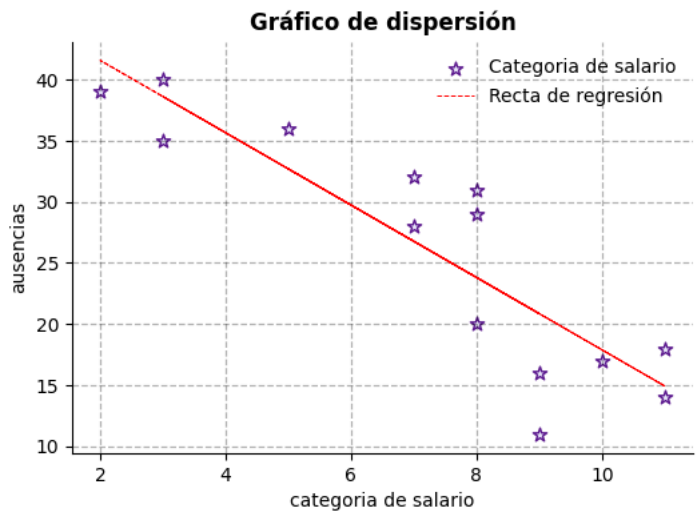
```
#6 Obtenga la recta de regresión ajustada
import statsmodels.api as sm
x_constante=sm.add_constant(x)
modelo=sm.OLS(y,x_constante).fit()
y_calculada=modelo.predict(x_constante)
modelo.params
```

	0
const	47.487603
categoria_salario	-2.958678

dtype: float64

```
from matplotlib.lines import lineStyles
import matplotlib.pyplot as plt
#configuración general
plt.figure(
    figsize=(6,4),  #tamaño (ancho, alto)
    dpi=100         #resolución
)
#Gráfico de dispersión
plt.scatter(
    x,y,
    marker="*",
    color="lavender",
    edgecolor="indigo", #color de borde
    alpha=0.9, #transparencia
    s=60, #tamaño de punto
    label="Categoria de salario" #etiqueta
)
plt.plot(
    x, y_calculada,
    color='red',   # color de la línea
    linewidth=0.7,        # grosor de la línea
    linestyle='--',        # estilo de línea
    marker='o',           # marcador en cada punto
    markersize=0,         # tamaño del marcador
    markerfacecolor='red',
    markeredgecolor='black',
    label='Recta de regresión'
)
#Título
plt.title(
    "Gráfico de dispersión",
    fontsize=12, #tamaño de fuente
    fontweight="bold" #bold:negritas
)
#Etiquetas eje x
plt.xlabel(
    "categoria de salario",
```

```
    fontsize=10,
)
#Etiqueta eje y
plt.ylabel(
    "ausencias",
    fontsize=10
)
#tamaño de los tricks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

#Márgenes
plt.margins(x=0.05,y=0.05)
plt.gca().spines[["top","right"]].set_visible(False)

plt.grid(
    visible=True,
    linestyle="--",
    linewidth=0.9, #ancho de line
    alpha=0.3, #transparencia
    color="black"
)
    #Leyenda
plt.legend(
        fontsize=10,
        loc="best",
        frameon=False
        #bbox_to_anchor=(0.5.-0.15)
)
        #Guardar imagen
plt.savefig(
            "Dispersión",
            bbox_inches="tight"
        )
```




```
#7 Intervalo de confianza
modelo.conf_int(alpha=0.05)
```

	0	1
const	39.707750	55.267457
categoria_salario	-3.962455	-1.954900

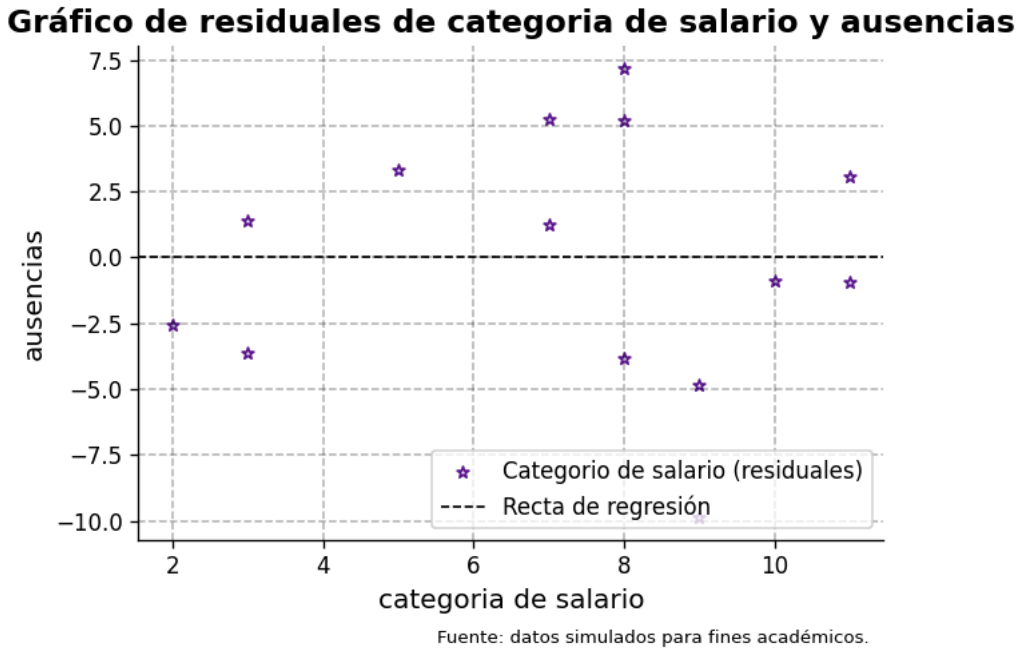
```
# 8. Calcule los residuales y trace un nuevo gráfico de dispersión. Comente,

# Los residuales son la diferencia entre el valor real y el valor que da el
# modelo

#1 residuales = y - y_calculada
residuales = modelo.resid

residuales
```

	0
0	3.057851
1	-0.900826
2	5.181818
3	3.305785
4	-9.859504
5	1.223140
6	-3.611570
7	-0.942149
8	-3.818182
9	5.223140
10	-2.570248
11	-4.859504
12	7.181818
13	1.388430

dtype: float64

```
# @title
# 2. Realice un diagrama de dispersión para estos datos.
import matplotlib.pyplot as plt

# --- Configuración general del gráfico ---
plt.figure(
    figsize=(6, 4),   # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120           # resolución del gráfico
)

# --- Gráfico de dispersión ---
plt.scatter(
    x, residuales,
    marker="*",       # forma
    color='lavender',     # color de los puntos
    edgecolor='indigo',    # borde de los puntos
    alpha=0.9,            # transparencia
    s=30,                 # tamaño de los puntos
    label='Categorio de salario (residuales)' # etiqueta para la leyenda
)

plt.axhline(
    y=0,     # Donde está la línea horizontal
    label="Recta de regresión", # Etiqueta
    linestyle = "--", # Estilo de línea
    color = "black",  # color
    linewidth=1.0,   # ancho de línea
    )

# --- Gráfico de línea ---
# plt.plot(
#     x, y_calculada,
#     color='black',   # color de la línea
#     linewidth=1.0,        # grosor de la línea
#     linestyle='--',        # estilo de línea
#     marker='o',           # marcador en cada punto
#     markersize=0,         # tamaño del marcador
#     markerfacecolor='white',
#     markeredgecolor='black',
#     label='Recta de regresión'
# )

# --- Título ---
plt.title(
    'Gráfico de residuales de categoria de salario y ausencias',
    fontsize=14,
    fontweight='bold'
)

# --- Etiquetas de los ejes ---
plt.xlabel(
    'categoria de salario',
```

```
    fontsize=12
)

plt.ylabel(
    'ausencias',
    fontsize=12
)

# --- Fuente de los ticks ---
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# --- Márgenes ---
plt.margins(x=0.05, y=0.05)  # espacio extra alrededor de los datos
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Para eliminar márgenes completamente, usar:
# plt.margins(0)

# --- Cuadrícula (opcional, pero didáctica) ---
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.9,
    alpha=0.3,
    color="black"
)

# --- Leyenda ---
plt.legend(
    fontsize=10,
    loc='lower right', # best
    frameon=True
)

# --- Nota al pie ---
plt.text(
    0.4, -0.2,
    'Fuente: datos simulados para fines académicos.',
    fontsize=8,
    ha='left',
    va='center',
    transform=plt.gca().transAxes
)

# --- Guardar gráfico ---
plt.savefig(
    "grafico de dispersión",
    bbox_inches='tight'
    )

plt.show()
```

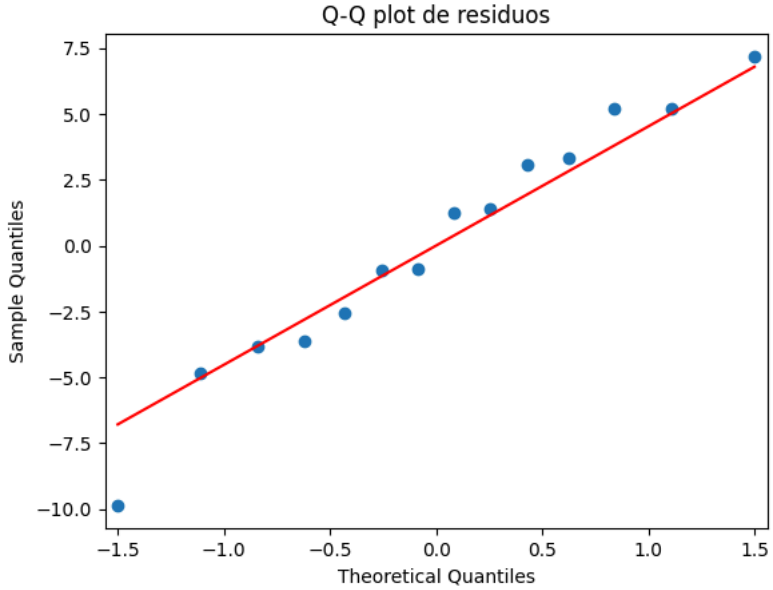


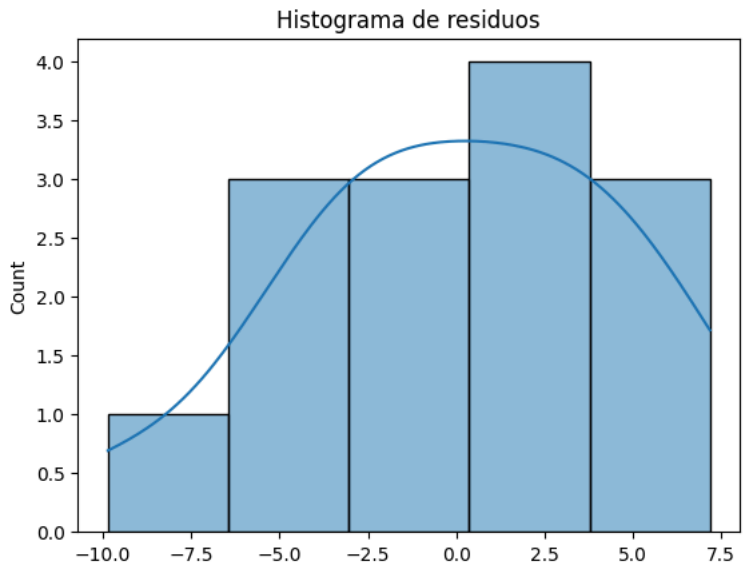

```
# 9. Realice la prueba de Shapiro para los residuales y comente el resultado.

# Test de Shapiro-Wilk
# H0: Hay normalidad
# H1: No hay normalidad
# alpha = 0.05

from scipy.stats import shapiro

_, valor_p = shapiro(residuales)

print(f"Valor-p: {valor_p: 0.4f}")
```

```
Valor-p:  0.8933
```

```
from scipy.stats import shapiro
import matplotlib.pyplot as plt
import seaborn as sns

# Prueba de Shapiro-Wilk
stat, valor_p_sh = shapiro(residuales)
print(f"valor-p (Shapiro) = {valor_p_sh}")

# Visualización: Q-Q plot
sm.qqplot(residuales, line='s')
plt.title("Q-Q plot de residuos")
plt.show()

# Histograma
sns.histplot(residuales, kde=True)
plt.title("Histograma de residuos")
plt.show()
```

```
valor-p (Shapiro) = 0.8932566424098969
```






```
# 10. Realice la prueba de Breusch-Pagan para los residuales
# y comente el resultado.

# Test de Breusch-Pagan
# H0: No hay heterocedasticidad (dispersión es igual en toda la recta)
# H1: Hay Heterocedasticidad (dispersión distinta a lo largo de la recta)
# alpha = 0.05

from statsmodels.stats.api import het_breuschpagan
_, valor_p_bp, _, _ = het_breuschpagan(residuales, x_constante)
print(f'valor_p de Breusch-Pagan: {valor_p_bp: 0.4f}\n')
```

```
valor_p de Breusch-Pagan:  0.4505
```

```
df.describe()
```

	categoria_salario	ausencias
count	14.000000	14.000000
mean	7.214286	26.142857
std	2.939874	9.882830
min	2.000000	11.000000
25%	5.500000	17.250000
50%	8.000000	28.500000
75%	9.000000	34.250000
max	11.000000	40.000000

```
modelo.predict([1, 60])
```

```
array([-130.03305785])
```

```
df
```

	categoria_salario	ausencias
0	11	18
1	10	17
2	8	29
3	5	36
4	9	11
5	7	28
6	3	35
7	11	14
8	8	20
9	7	32
10	2	39
11	9	16
12	8	31
13	3	40

Próximos pasos: Generar código con df New interactive sheet

```
modelo.params
```

	0
const	47.487603
categoria_salario	-2.958678

dtype: float64

```
# 12. Realice una tabla ANOVA e interprete el resultado.

# test de ANOVA (Analysis of Variance)
# H0: beta_1 = 0   (No hay correlación)
# H1: beta_1 ≠ 0   (Sí hay correlación)

from statsmodels.formula.api import ols
# Y ~ X
modelo_lineal = ols('categoria_salario ~ ausencias', data = df).fit()
tabla_anova = sm.stats.anova_lm(modelo_lineal)
tabla_anova
```

	df	sum_sq	mean_sq	F	PR(>F)
ausencias	1.0	87.034348	87.034348	41.243954	0.000033
Residual	12.0	25.322795	2.110233	NaN	NaN

Próximos pasos: Generar código con tabla_anova New interactive sheet

Problema 3

A menudo, quienes hacen la contabilidad de costos estiman los gastos generales con base en el nivel de producción. En Standard Knitting Co. han reunido información acerca de los gastos generales y las unidades producidas en diferentes plantas.

Gastos generales	191	170	272	155	280	173	234	116	153	178
Unidades	40	42	53	35	56	39	48	30	37	40

1. Establesca una variable dependiente (Y) y una variable independiente (X).
2. Realice un diagrama de dispersión para estos datos.
3. ¿Los datos soportan la suposición de linealidad?
4. Calcule el coeficiente de correlación e interprete el resultado.
5. Calcule el coeficiente de determinación e interprete el resultado.
6. Obtenga la recta de regresión ajustada y grafíquelo sobre el gráfico de dispersión.
7. Obtenga un intervalo de confianza del 95% para la pendiente de la recta de regresión ajustada (b_1)
8. Calcule los residuales y trace un nuevo gráfico de dispersión. Comente, ¿Parece que se verifican los supuestos?
9. Realice la prueba de Shapiro para los residuales y comente el resultado.
10. Realice la prueba de Brausch-Pagan para los residuales y comente el resultado.
11. Utiliza la recta de regresión para interpolar dos valores y extrapolar uno. Comenta estos resultados.
12. Realice una tabla ANOVA e interprete el resultado.

```
import pandas as pd

df = pd.DataFrame({
    "gasto_general": [191,170,272,155,280,173,234,116,153,178],
    "unidades": [40,42,53,35,56,39,48,30,37,40]
})

df
```

	gasto_general	unidades
0	191	40
1	170	42
2	272	53
3	155	35
4	280	56
5	173	39
6	234	48
7	116	30
8	153	37
9	178	40

Próximos pasos: Generar código con df New interactive sheet

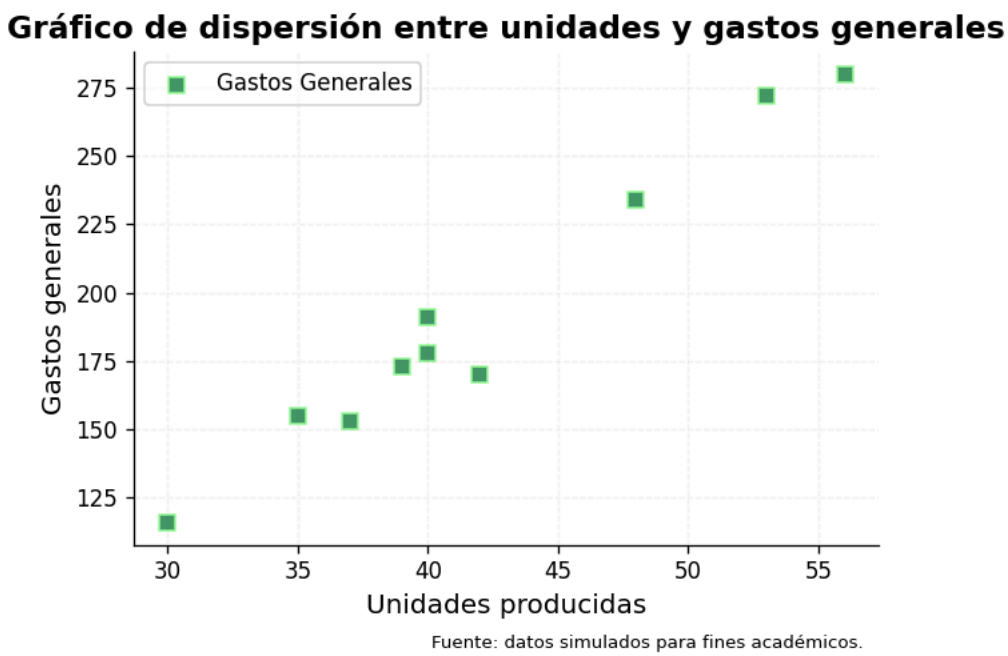
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 2 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   gasto_general  10 non-null     int64
 1   unidades       10 non-null     int64
dtypes: int64(2)
memory usage: 292.0 bytes
```

```
# 1. Establesca una variable dependiente ( Y )
# y una variable independiente ( X ).

x = df["unidades"]             # variable independiente (con la que se cuenta)
y = df["gasto_general"]        # variable de interés (variable dependiente)
```

```
# @title
# 2. Realice un diagrama de dispersión para estos datos.
import matplotlib.pyplot as plt

# --- Configuración general del gráfico ---
plt.figure(
    figsize=(6, 4),   # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120           # resolución del gráfico
)

# --- Gráfico de dispersión ---
plt.scatter(
    x, y,
    marker="s",       # forma
    color='seagreen',     # color de los puntos
    edgecolor='palegreen',    # borde de los puntos
    alpha=0.9,            # transparencia
    s=50,                 # tamaño de los puntos
    label='Gastos Generales' # etiqueta para la leyenda
)

# --- Gráfico de línea ---
# plt.plot(
#     x, y,
#     color='orange',   # color de la línea
#     linewidth=2.2,        # grosor de la línea
#     linestyle='-',        # estilo de línea
#     marker='o',           # marcador en cada punto
#     markersize=5,         # tamaño del marcador
#     markerfacecolor='white',
#     markeredgecolor='black',
#     label='Datos observados'
# )

# --- Título ---
plt.title(
    'Gráfico de dispersión entre unidades y gastos generales',
    fontsize=14,
    fontweight='bold'
)

# --- Etiquetas de los ejes ---
plt.xlabel(
    'Unidades producidas',
    fontsize=12
)

plt.ylabel(
    'Gastos generales',
    fontsize=12
)

# --- Fuente de los ticks ---
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# --- Márgenes ---
plt.margins(x=0.05, y=0.05)  # espacio extra alrededor de los datos
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Para eliminar márgenes completamente, usar:
# plt.margins(0)

# --- Cuadrícula (opcional, pero didáctica) ---
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.1,
    color="gray"
)

# --- Leyenda ---
plt.legend(
    fontsize=10,
    loc='best',
    frameon=True
)
```

```
# --- Nota al pie ---
plt.text(
    0.4, -0.2,
    'Fuente: datos simulados para fines académicos.',
    fontsize=8,
    ha='left',
    va='center',
    transform=plt.gca().transAxes
)

# --- Guardar gráfico ---
plt.savefig(
    "grafico de dispersión",
    bbox_inches='tight'
    )

plt.show()
```



3. Soporta la suposición de linealidad? Sí.

```
#4. Calcule el coeficiente de correlación e interprete el resultado.
from scipy.stats import pearsonr

# Test de Pearson
# H0: rho = 0     (No hay correlación)
# H1: rho ≠ 0     (Sí hay correlación)
# alpha = 0.05

r, valor_p = pearsonr(x, y)

print(f"Coeficiente de correlación: {r: 0.4f}")
print(f'Valor p: {valor_p: 0.4f}')
```

```
Coeficiente de correlación:  0.9835
Valor p:  0.0000
```

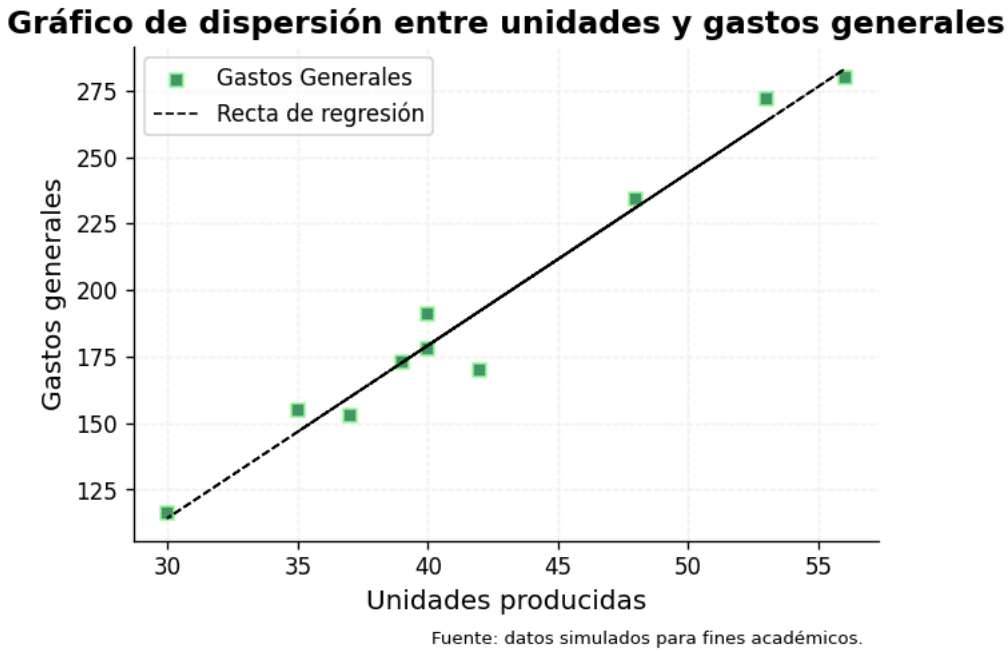
```
# 6. Obtenga la recta de regresión ajustada y grafíquelo sobre el
# gráfico de dispersión

import statsmodels.api as sm
x_constante = sm.add_constant(x)
modelo = sm.OLS(y, x_constante).fit()
y_calculada = modelo.predict(x_constante)

modelo.params
```

	0
const	-80.442857
unidades	6.491497

dtype: float64

```
# @title
# 2. Realice un diagrama de dispersión para estos datos.
import matplotlib.pyplot as plt

# --- Configuración general del gráfico ---
plt.figure(
    figsize=(6, 4),   # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120           # resolución del gráfico
)

# --- Gráfico de dispersión ---
plt.scatter(
    x, y,
    marker="s",       # forma
    color='seagreen',     # color de los puntos
    edgecolor='palegreen',    # borde de los puntos
    alpha=0.9,            # transparencia
    s=30,                 # tamaño de los puntos
    label='Gastos Generales' # etiqueta para la leyenda
)

# --- Gráfico de línea ---
plt.plot(
    x, y_calculada,
    color='black',   # color de la línea
    linewidth=1.0,        # grosor de la línea
    linestyle='--',        # estilo de línea
    marker='o',           # marcador en cada punto
    markersize=0,         # tamaño del marcador
    markerfacecolor='white',
    markeredgecolor='black',
    label='Recta de regresión'
)

# --- Título ---
plt.title(
    'Gráfico de dispersión entre unidades y gastos generales',
    fontsize=14,
    fontweight='bold'
)

# --- Etiquetas de los ejes ---
plt.xlabel(
    'Unidades producidas',
    fontsize=12
)

plt.ylabel(
    'Gastos generales',
    fontsize=12
)

# --- Fuente de los ticks ---
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# --- Márgenes ---
plt.margins(x=0.05, y=0.05)  # espacio extra alrededor de los datos
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Para eliminar márgenes completamente, usar:
# plt.margins(0)

# --- Cuadrícula (opcional, pero didáctica) ---
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.1,
    color="gray"
```

```
)

# --- Leyenda ---
plt.legend(
    fontsize=10,
    loc='best',
    frameon=True
)

# --- Nota al pie ---
plt.text(
    0.4, -0.2,
    'Fuente: datos simulados para fines académicos.',
    fontsize=8,
    ha='left',
    va='center',
    transform=plt.gca().transAxes
)

# --- Guardar gráfico ---
plt.savefig(
    "grafico de dispersión",
    bbox_inches='tight'
    )

plt.show()
```




```
# 5. Calcule el coeficiente de determinación e interprete el resultado.
from sklearn.metrics import r2_score  # recomendada
r2 = r2_score(y, y_calculada)
print(f'Coeficiente de determinación: {r2: 0.2%}\n')
```

```
Coeficiente de determinación:  96.73%
```

```
modelo.params
```

	0
const	-80.442857
unidades	6.491497

dtype: float64

```
# 7. Obtenga un intervalo de confianza del 95% para la pendiente de la recta de
# regresión ajustada ( b1 )

# intervalo de confianza = 1 - alpha

modelo.conf_int(alpha = 0.05)
```

	0	1
const	-121.986373	-38.899341
unidades	5.518450	7.464543

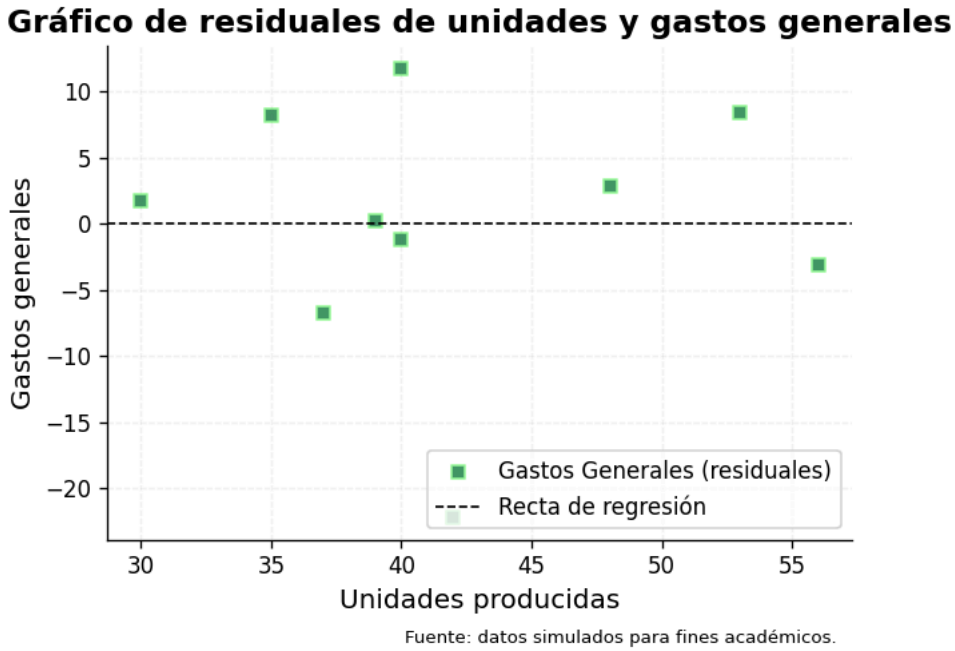
```
# 8. Calcule los residuales y trace un nuevo gráfico de dispersión. Comente,
# ¿Parece que se verifican los supuestos?

# Los residuales son la diferencia entre el valor real y el valor que da el
# modelo

# residuales = y - y_calculada
residuales = modelo.resid

residuales
```

	0
0	11.782993
1	-22.200000
2	8.393537
3	8.240476
4	-3.080952
5	0.274490
6	2.851020
7	1.697959
8	-6.742517
9	-1.217007

dtype: float64

```
# @title
# 2. Realice un diagrama de dispersión para estos datos.
import matplotlib.pyplot as plt

# --- Configuración general del gráfico ---
plt.figure(
    figsize=(6, 4),   # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120           # resolución del gráfico
)

# --- Gráfico de dispersión ---
plt.scatter(
    x, residuales,
    marker="s",       # forma
    color='seagreen',     # color de los puntos
    edgecolor='palegreen',    # borde de los puntos
    alpha=0.9,            # transparencia
    s=30,                 # tamaño de los puntos
    label='Gastos Generales (residuales)' # etiqueta para la leyenda
)

plt.axhline(
    y=0,     # Donde está la línea horizontal
    label="Recta de regresión", # Etiqueta
    linestyle = "--", # Estilo de línea
    color = "black",  # color
    linewidth=1.0,   # ancho de línea
    )

# --- Gráfico de línea ---
# plt.plot(
#     x, y_calculada,
#     color='black',   # color de la línea
#     linewidth=1.0,        # grosor de la línea
#     linestyle='--',        # estilo de línea
#     marker='o',           # marcador en cada punto
#     markersize=0,         # tamaño del marcador
#     markerfacecolor='white',
#     markeredgecolor='black',
#     label='Recta de regresión'
```

```
# )

# --- Título ---
plt.title(
    'Gráfico de residuales de unidades y gastos generales',
    fontsize=14,
    fontweight='bold'
)

# --- Etiquetas de los ejes ---
plt.xlabel(
    'Unidades producidas',
    fontsize=12
)

plt.ylabel(
    'Gastos generales',
    fontsize=12
)

# --- Fuente de los ticks ---
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# --- Márgenes ---
plt.margins(x=0.05, y=0.05)  # espacio extra alrededor de los datos
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Para eliminar márgenes completamente, usar:
# plt.margins(0)

# --- Cuadrícula (opcional, pero didáctica) ---
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.1,
    color="gray"
)

# --- Leyenda ---
plt.legend(
    fontsize=10,
    loc='lower right', # best
    frameon=True
)

# --- Nota al pie ---
plt.text(
    0.4, -0.2,
    'Fuente: datos simulados para fines académicos.',
    fontsize=8,
    ha='left',
    va='center',
    transform=plt.gca().transAxes
)

# --- Guardar gráfico ---
plt.savefig(
    "grafico de dispersión",
    bbox_inches='tight'
    )

plt.show()
```

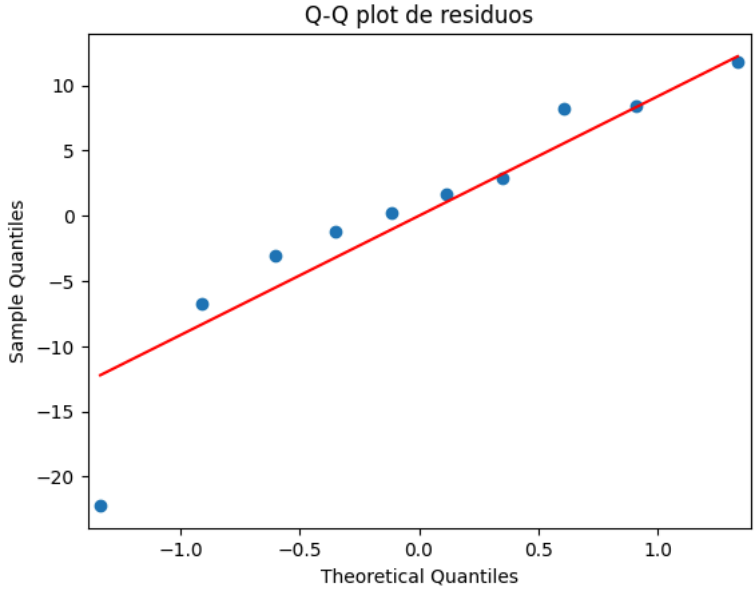


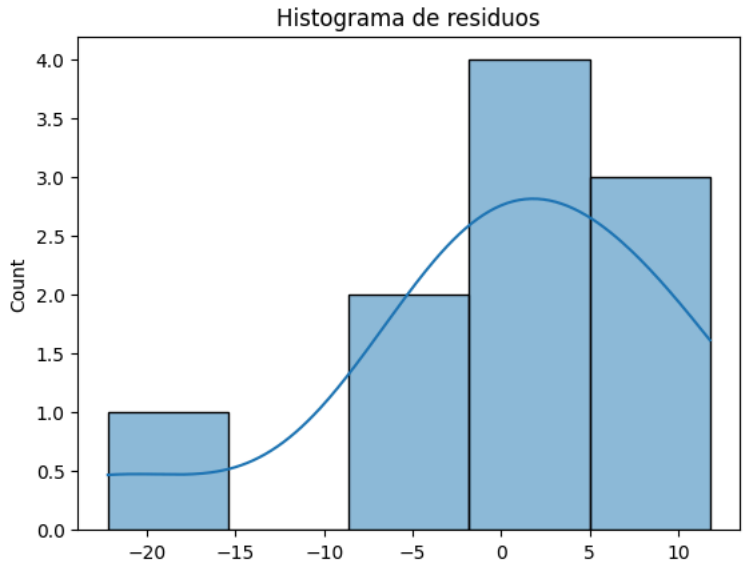

```
# 9. Realice la prueba de Shapiro para los residuales y comente el resultado.

# Test de Shapiro-Wilk
# H0: Hay normalidad
# H1: No hay normalidad
# alpha = 0.05

from scipy.stats import shapiro

_, valor_p = shapiro(residuales)

print(f"Valor-p: {valor_p: 0.4f}")
```

```
Valor-p:  0.2205
```

```
from scipy.stats import shapiro
import matplotlib.pyplot as plt
import seaborn as sns

# Prueba de Shapiro-Wilk
stat, valor_p_sh = shapiro(residuales)
print(f"valor-p (Shapiro) = {valor_p_sh}")

# Visualización: Q-Q plot
sm.qqplot(residuales, line='s')
plt.title("Q-Q plot de residuos")
plt.show()

# Histograma
sns.histplot(residuales, kde=True)
plt.title("Histograma de residuos")
plt.show()
```

```
valor-p (Shapiro) = 0.22045077017698533
```






```
# 10. Realice la prueba de Breusch-Pagan para los residuales
# y comente el resultado.

# Test de Breusch-Pagan
# H0: No hay heterocedasticidad (dispersión es igual en toda la recta)
# H1: Hay Heterocedasticidad (dispersión distinta a lo largo de la recta)
# alpha = 0.05

from statsmodels.stats.api import het_breuschpagan
_, valor_p_bp, _, _ = het_breuschpagan(residuales, x_constante)
print(f'valor_p de Breusch-Pagan: {valor_p_bp: 0.4f}\n')
```

```
valor_p de Breusch-Pagan:  0.9858
```

```
df.describe()
```

	gasto_general	unidades
count	10.000000	10.000000
mean	192.200000	42.000000
std	53.349581	8.082904
min	116.000000	30.000000
25%	158.750000	37.500000
50%	175.500000	40.000000
75%	223.250000	46.500000
max	280.000000	56.000000

```
modelo.predict([1, 60])
```

```
array([309.04693878])
```

```
df
```

	gasto_general	unidades
0	191	40
1	170	42
2	272	53
3	155	35
4	280	56
5	173	39
6	234	48
7	116	30
8	153	37
9	178	40

Próximos pasos: Generar código con df | New interactive sheet

```
modelo.params
```

	0
const	-80.442857
unidades	6.491497

dtype: float64

```
# 12. Realice una tabla ANOVA e interprete el resultado.

# test de ANOVA (Analysis of Variance)
# H0: beta_1 = 0   (No hay correlación)
# H1: beta_1 ≠ 0   (Sí hay correlación)

from statsmodels.formula.api import ols
# Y ~ X
modelo_lineal = ols('gasto_general ~ unidades', data = df).fit()
tabla_anova = sm.stats.anova_lm(modelo_lineal)
tabla_anova
```

	df	sum_sq	mean_sq	F	PR(>F)
unidades	1.0	24778.042517	24778.042517	236.669535	3.167080e-07
Residual	8.0	837.557483	104.694685	NaN	NaN

Próximos pasos: Generar código con tabla_anova | New interactive sheet

Problema 4

Las ventas de línea blanca varían según el estado del mercado de casas nuevas: cuando las ventas de casas nuevas son buenas, también lo son las de lavaplatos, lavadoras de ropa, secadoras y refrigeradores. Una asociación de comercio compiló los siguientes datos históricos (en miles de unidades) de las ventas de línea blanca y la construcción de casas.

Construcción de casas (miles)	Ventas de línea blanca (miles)
2.0	5.0
2.5	5.5
3.2	6.0
3.6	7.0
3.7	7.2
4.0	7.7
4.2	8.4
4.6	9.0
4.8	9.7
5.0	10.0

1. Establesca una variable dependiente (Y) y una variable independiente (X).
2. Realice un diagrama de dispersión para estos datos.
3. ¿Los datos soportan la suposición de linealidad?
4. Calcule el coeficiente de correlación e interprete el resultado.
5. Calcule el coeficiente de determinación e interprete el resultado.
6. Obtenga la recta de regresión ajustada y grafíquelo sobre el gráfico de dispersión.
7. Obtenga un intervalo de confianza del 95% para la pendiente de la recta de regresión ajustada (b_1)
8. Calcule los residuales y trace un nuevo gráfico de dispersión. Comente, ¿Parece que se verifican los supuestos?
9. Realice la prueba de Shapiro para los residuales y comente el resultado.
10. Realice la prueba de Brausch-Pagan para los residuales y comente el resultado.
11. Utiliza la recta de regresión para interpolar dos valores y extrapolar uno. Comenta estos resultados.
12. Realice una tabla ANOVA e interprete el resultado.

```
import pandas as pd

df = pd.DataFrame({
    "construccion_de_casas": [2.0,2.5,3.2,3.6,3.7,4.0,4.2,4.6,4.8,5.0],
    "ventas_de_linea_blanca": [5.0,5.5,6.0,7.0,7.2,7.7,8.4,9.0,9.7,10.0]
})

df
```

	construccion_de_casas	ventas_de_linea_blanca
0	2.0	5.0
1	2.5	5.5
2	3.2	6.0
3	3.6	7.0
4	3.7	7.2
5	4.0	7.7
6	4.2	8.4
7	4.6	9.0
8	4.8	9.7
9	5.0	10.0

Próximos pasos: Generar código con df New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 2 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   construccion_de_casas   10 non-null     float64
 1   ventas_de_linea_blanca  10 non-null     float64
dtypes: float64(2)
memory usage: 292.0 bytes
```

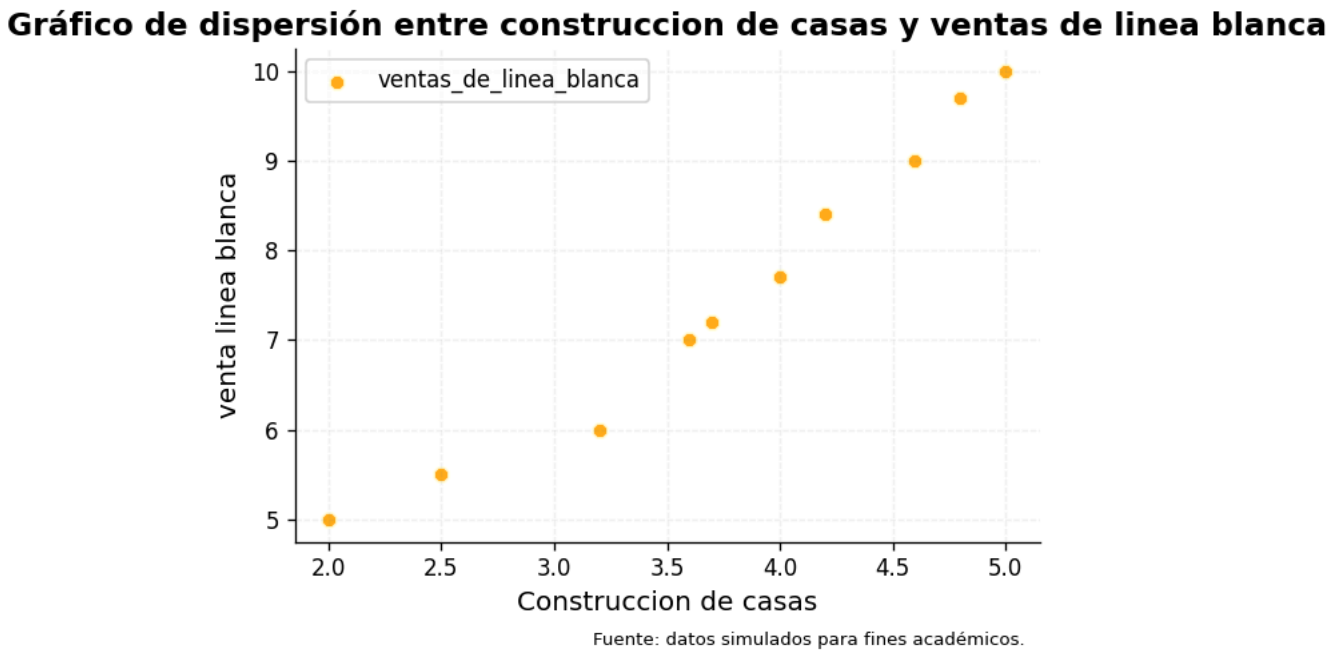
```
df.describe()
```

	construccion_de_casas	ventas_de_linea_blanca
count	10.000000	10.000000
mean	3.760000	7.550000
std	0.977753	1.732211
min	2.000000	5.000000
25%	3.300000	6.250000
50%	3.850000	7.450000
75%	4.500000	8.850000
max	5.000000	10.000000

```
# 1. Establesca una variable dependiente ( Y )
# y una variable independiente ( X ).

x = df["construccion_de_casas"]             # variable independiente (con la que se cuenta)
y = df["ventas_de_linea_blanca"]        # variable de interés (variable dependiente)
```

```
# @title
# 2. Realice un diagrama de dispersión para estos datos.
import matplotlib.pyplot as plt

# --- Configuración general del gráfico ---
plt.figure(
    figsize=(6, 4),   # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120           # resolución del gráfico
)

# --- Gráfico de dispersión ---
plt.scatter(
    x, y,
    marker="8",       # forma
    color='orange',     # color de los puntos
    edgecolor='lightyellow',    # borde de los puntos
    alpha=0.9,            # transparencia
    s=50,                 # tamaño de los puntos
    label='ventas_de_linea_blanca' # etiqueta para la leyenda
)

# --- Gráfico de línea ---
# plt.plot(
#     x, y,
#     color='orange',   # color de la línea
#     linewidth=2.2,        # grosor de la línea
#     linestyle='-',        # estilo de línea
#     marker='o',           # marcador en cada punto
#     markersize=5,         # tamaño del marcador
#     markerfacecolor='white',
#     markeredgecolor='black',
#     label='Datos observados'
# )

# --- Título ---
plt.title(
    'Gráfico de dispersión entre construccion de casas y ventas de linea blanca',
    fontsize=14,
    fontweight='bold'
)

# --- Etiquetas de los ejes ---
plt.xlabel(
    'Construccion de casas',
    fontsize=12
)

plt.ylabel(
    'venta linea blanca',
    fontsize=12
)

# --- Fuente de los ticks ---
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)
```

```
# --- Márgenes ---
plt.margins(x=0.05, y=0.05)  # espacio extra alrededor de los datos
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Para eliminar márgenes completamente, usar:
# plt.margins(0)

# --- Cuadrícula (opcional, pero didáctica) ---
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.1,
    color="gray"
)

# --- Leyenda ---
plt.legend(
    fontsize=10,
    loc='best',
    frameon=True
)

# --- Nota al pie ---
plt.text(
    0.4, -0.2,
    'Fuente: datos simulados para fines académicos.',
    fontsize=8,
    ha='left',
    va='center',
    transform=plt.gca().transAxes
)

# --- Guardar gráfico ---
plt.savefig(
    "grafico de dispersión",
    bbox_inches='tight'
    )

plt.show()
```




```
#4.Calcule el coeficiente de correlación e interprete el resultado.
from scipy.stats import pearsonr

# Test de Pearson
# H0: rho = 0     (No hay correlación)
# H1: rho ≠ 0     (Sí hay correlación)
# alpha = 0.05

r, valor_p = pearsonr(x, y)
```

```
print(f"Coeficiente de correlación: {r: 0.4f}")
print(f'Valor p: {valor_p: 0.4f}')
```

```
Coeficiente de correlación:  0.9808
Valor p:  0.0000
```

```
# 5. Calcule el coeficiente de determinación e interprete el resultado.
from sklearn.metrics import r2_score  # recomendada
r2 = r2_score(y, y_calculada)
print(f'Coeficiente de determinación: {r2: 0.2%}\n')
```

```
Coeficiente de determinación: -1356310.92%
```

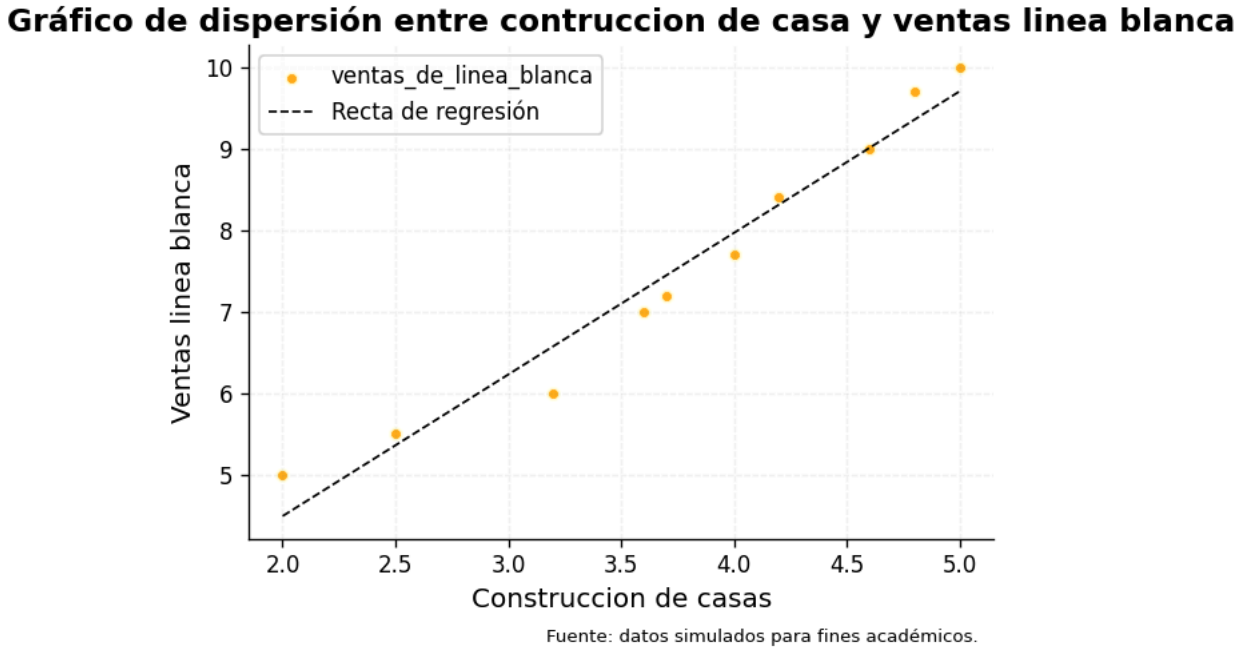
```
# 6. Obtenga la recta de regresión ajustada y grafíquelo sobre el
# gráfico de dispersión

import statsmodels.api as sm
x_constante = sm.add_constant(x)
modelo = sm.OLS(y, x_constante).fit()
y_calculada = modelo.predict(x_constante)

modelo.params
```

	0
const	1.016760
construccion_de_casas	1.737564

dtype: float64

```
# @title
# 2. Realice un diagrama de dispersión para estos datos.
import matplotlib.pyplot as plt

# --- Configuración general del gráfico ---
plt.figure(
    figsize=(6, 4),   # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120           # resolución del gráfico
)

# --- Gráfico de dispersión ---
plt.scatter(
    x, y,
    marker="8",       # forma
    color='orange',     # color de los puntos
    edgecolor='lightyellow',    # borde de los puntos
    alpha=0.9,            # transparencia
    s=30,                 # tamaño de los puntos
    label='ventas_de_linea_blanca' # etiqueta para la leyenda
)

# --- Gráfico de línea ---
plt.plot(
    x, y_calculada,
    color='black',   # color de la línea
    linewidth=1.0,        # grosor de la línea
    linestyle='--',        # estilo de línea
    marker='o',           # marcador en cada punto
    markersize=0,         # tamaño del marcador
    markerfacecolor='white',
    markeredgecolor='black',
    label='Recta de regresión'
)

# --- Título ---
plt.title(
    'Gráfico de dispersión entre contruccion de casa y ventas linea blanca',
    fontsize=14,
    fontweight='bold'
)

# --- Etiquetas de los ejes ---
plt.xlabel(
    'Construccion de casas',
    fontsize=12
)
```

```
plt.ylabel(
    'Ventas linea blanca',
    fontsize=12
)

# --- Fuente de los ticks ---
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# --- Márgenes ---
plt.margins(x=0.05, y=0.05)  # espacio extra alrededor de los datos
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Para eliminar márgenes completamente, usar:
# plt.margins(0)

# --- Cuadrícula (opcional, pero didáctica) ---
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.1,
    color="gray"
)

# --- Leyenda ---
plt.legend(
    fontsize=10,
    loc='best',
    frameon=True
)

# --- Nota al pie ---
plt.text(
    0.4, -0.2,
    'Fuente: datos simulados para fines académicos.',
    fontsize=8,
    ha='left',
    va='center',
    transform=plt.gca().transAxes
)

# --- Guardar gráfico ---
plt.savefig(
    "grafico de dispersión",
    bbox_inches='tight'
    )

plt.show()
```



```
modelo.params
```

	0
const	1.016760
construccion_de_casas	1.737564

dtype: float64

```
# 7. Obtenga un intervalo de confianza del 95% para la pendiente de la recta de
# regresión ajustada ( b1 )

# intervalo de confianza = 1 - alpha

modelo.conf_int(alpha = 0.05)
```

	0	1
const	-0.074848	2.108368
construccion_de_casas	1.455693	2.019435

```
# 8. Calcule los residuales y trace un nuevo gráfico de dispersión. Comente,
# ¿Parece que se verifican los supuestos?

# Los residuales son la diferencia entre el valor real y el valor que da el
# modelo

# residuales = y - y_calculada
residuales = modelo.resid

residuales
```

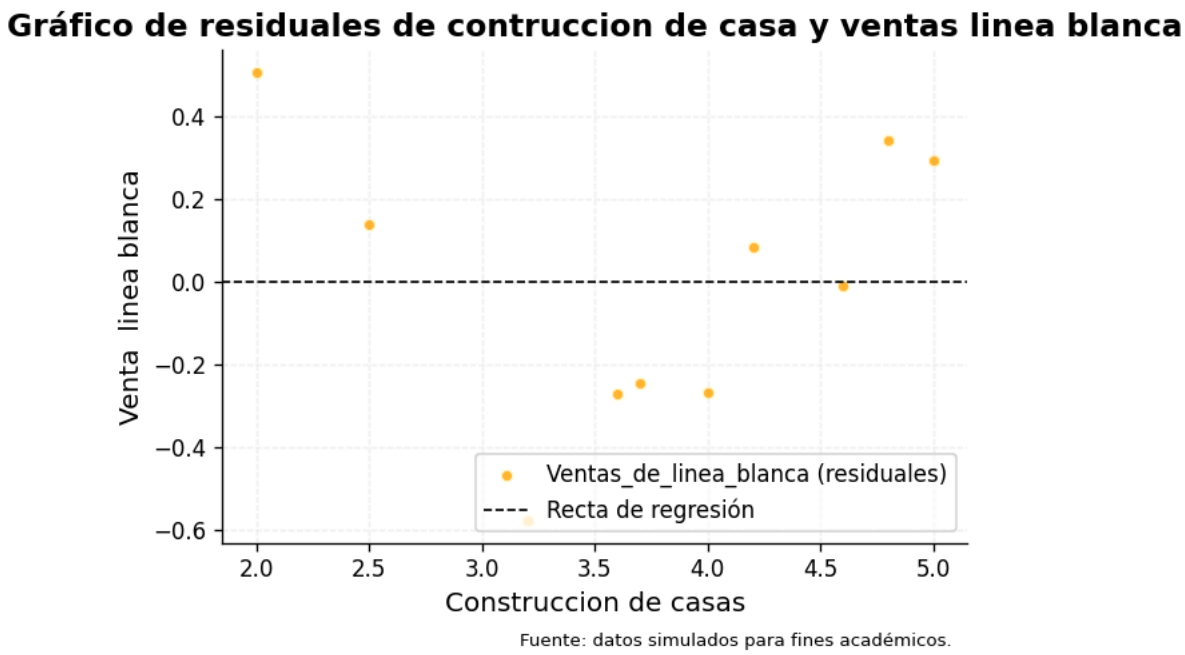
	0
0	0.508113
1	0.139331
2	-0.576964
3	-0.271990
4	-0.245746
5	-0.267015
6	0.085472
7	-0.009554
8	0.342934
9	0.295421

dtype: float64

```
# @title
# 2. Realice un diagrama de dispersión para estos datos.
import matplotlib.pyplot as plt

# --- Configuración general del gráfico ---
plt.figure(
    figsize=(6, 4),   # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120           # resolución del gráfico
)

# --- Gráfico de dispersión ---
plt.scatter(
    x, residuales,
    marker="8",       # forma
    color='orange',     # color de los puntos
    edgecolor='lightyellow',    # borde de los puntos
    alpha=0.8,            # transparencia
    s=30,                 # tamaño de los puntos
    label='Ventas_de_linea_blanca (residuales)' # etiqueta para la leyenda
)

plt.axhline(
    y=0,     # Donde está la línea horizontal
```

```
    label="Recta de regresión", # Etiqueta
    linestyle = "--", # Estilo de línea
    color = "black",  # color
    linewidth=1.0,   # ancho de línea
    )

# --- Gráfico de línea ---
# plt.plot(
#     x, y_calculada,
#     color='black',   # color de la línea
#     linewidth=1.0,        # grosor de la línea
#     linestyle='--',        # estilo de línea
#     marker='o',           # marcador en cada punto
#     markersize=0,         # tamaño del marcador
#     markerfacecolor='white',
#     markeredgecolor='black',
#     label='Recta de regresión'
# )

# --- Título ---
plt.title(
    'Gráfico de residuales de contruccion de casa y ventas linea blanca',
    fontsize=14,
    fontweight='bold'
)

# --- Etiquetas de los ejes ---
plt.xlabel(
    'Construccion de casas',
    fontsize=12
)

plt.ylabel(
    'Venta  linea blanca',
    fontsize=12
)

# --- Fuente de los ticks ---
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# --- Márgenes ---
plt.margins(x=0.05, y=0.05)  # espacio extra alrededor de los datos
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Para eliminar márgenes completamente, usar:
# plt.margins(0)

# --- Cuadrícula (opcional, pero didáctica) ---
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.1,
    color="gray"
)

# --- Leyenda ---
plt.legend(
    fontsize=10,
    loc='lower right', # best
    frameon=True
)

# --- Nota al pie ---
plt.text(
    0.4, -0.2,
    'Fuente: datos simulados para fines académicos.',
    fontsize=8,
    ha='left',
    va='center',
    transform=plt.gca().transAxes
)

# --- Guardar gráfico ---
plt.savefig(
    "grafico de dispersión",
    bbox_inches='tight'
    )
```

```
plt.show()
```

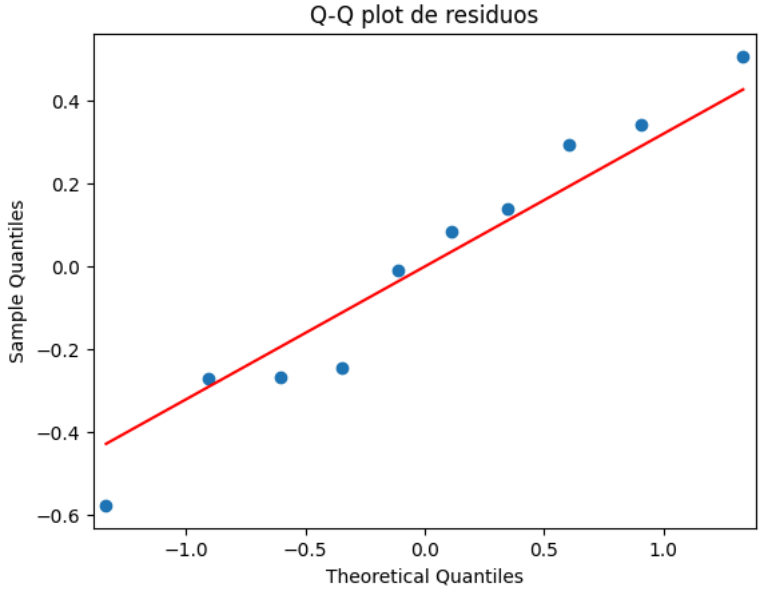


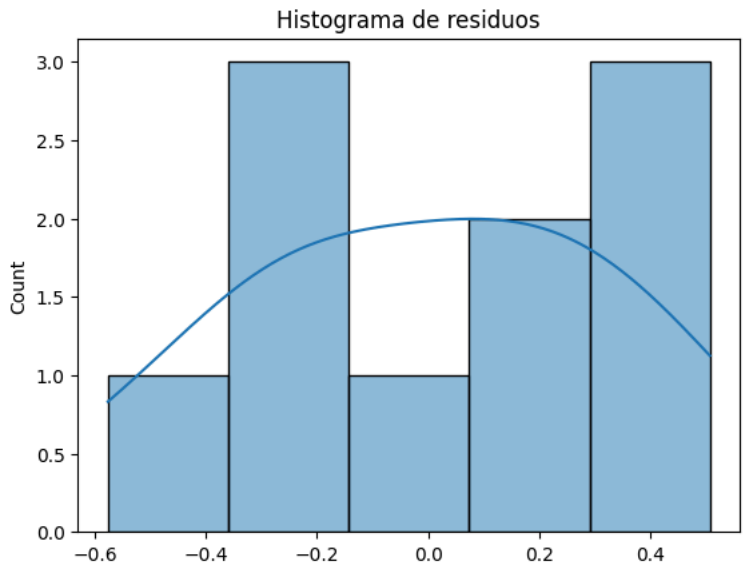

```
# 9. Realice la prueba de Shapiro para los residuales y comente el resultado.

# Test de Shapiro-Wilk
# H0: Hay normalidad
# H1: No hay normalidad
# alpha = 0.05

from scipy.stats import shapiro

_, valor_p = shapiro(residuales)

print(f"Valor-p: {valor_p: 0.4f}")
```

```
Valor-p:  0.8464
```

```
from scipy.stats import shapiro
import matplotlib.pyplot as plt
import seaborn as sns

# Prueba de Shapiro-Wilk
stat, valor_p_sh = shapiro(residuales)
print(f"valor-p (Shapiro) = {valor_p_sh}")

# Visualización: Q-Q plot
sm.qqplot(residuales, line='s')
plt.title("Q-Q plot de residuos")
plt.show()

# Histograma
sns.histplot(residuales, kde=True)
plt.title("Histograma de residuos")
plt.show()
```

```
valor-p (Shapiro) = 0.8463507249054649
```






```
# 10. Realice la prueba de Breusch-Pagan para los residuales
# y comente el resultado.

# Test de Breusch-Pagan
# H0: No hay heterocedasticidad (dispersión es igual en toda la recta)
# H1: Hay Heterocedasticidad (dispersión distinta a lo largo de la recta)
# alpha = 0.05

from statsmodels.stats.api import het_breuschpagan
_, valor_p_bp, _, _ = het_breuschpagan(residuales, x_constante)
print(f'valor_p de Breusch-Pagan: {valor_p_bp: 0.4f}\n')
```

```
valor_p de Breusch-Pagan:  0.1581
```

```
df.describe()
```

	construccion_de_casas	ventas_de_linea_blanca
count	10.000000	10.000000
mean	3.760000	7.550000
std	0.977753	1.732211
min	2.000000	5.000000
25%	3.300000	6.250000
50%	3.850000	7.450000
75%	4.500000	8.850000
max	5.000000	10.000000

```
# para intrepolado
modelo.predict([1, 2])
```

```
array([4.49188749])
```

esto es lo que se espera y:venda de linea blanca (unidades)

```
df
```

	construccion_de_casas	ventas_de_linea_blanca
0	2.0	5.0
1	2.5	5.5
2	3.2	6.0
3	3.6	7.0
4	3.7	7.2
5	4.0	7.7
6	4.2	8.4
7	4.6	9.0
8	4.8	9.7
9	5.0	10.0

Próximos pasos: Generar código con df | New interactive sheet

```
#11 para extrapolar
modelo.predict([1,6])
```

```
array([11.44214319])
```

Haz doble clic (o ingresa) para editar

```
modelo.params
```

	0
const	1.016760
construccion_de_casas	1.737564

dtype: float64

```
# 12. Realice una tabla ANOVA e interprete el resultado.

# test de ANOVA (Analysis of Variance)
# H0: beta_1 = 0   (No hay correlación)
# H1: beta_1 ≠ 0   (Sí hay correlación)

from statsmodels.formula.api import ols
# Y ~ X
modelo_lineal = ols('construccion_de_casas ~ ventas_de_linea_blanca ', data = df).fit()
tabla_anova = sm.stats.anova_lm(modelo_lineal)
tabla_anova
```

	df	sum_sq	mean_sq	F	PR(>F)
ventas_de_linea_blanca	1.0	8.276338	8.276338	202.069951	5.841003e-07
Residual	8.0	0.327662	0.040958	NaN	NaN

Próximos pasos: Generar código con `tabla_anova` New interactive sheet

soporta la suposicion de linealidad ? si

Problema 5

William C. Andrews, consultor de comportamiento organizacional de Victory Motorcycles, ha diseñado una prueba para mostrar a los supervisores de la compañía los peligros de sobrevigilar a sus trabajadores. Un trabajador de la línea de ensamble tiene a su cargo una serie de tareas complicadas. Durante el desempeño del trabajador, un inspector lo interrumpe constantemente para ayudarlo a terminar las tareas. El trabajador, después de terminar su trabajo, recibe una prueba psicológica diseñada para medir la hostilidad del trabajador hacia la autoridad (una alta puntuación implica una hostilidad baja). A ocho distintos trabajadores se les asignaron las tareas y luego se les interrumpió para darles instrucciones útiles un número variable de veces (línea X). Sus calificaciones en la prueba de hostilidad se dan en el renglón Y.

número interrupciones al trabajador	**5**	**10**	**10**	**15**	**15**	**20**	**20**	**25**
calificación del trabajador en la prueba de hostilidad	58	41	45	27	26	12	16	3

1. Establesca una variable dependiente (Y) y una variable independiente (X).
2. Realice un diagrama de dispersión para estos datos.
3. ¿Los datos soportan la suposición de linealidad?
4. Calcule el coeficiente de correlación e interprete el resultado.
5. Calcule el coeficiente de determinación e interprete el resultado.
6. Obtenga la recta de regresión ajustada y grafíquelo sobre el gráfico de dispersión.
7. Obtenga un intervalo de confianza del 95% para la pendiente de la recta de regresión ajustada (b_1)
8. Calcule los residuales y trace un nuevo gráfico de dispersión. Comente, ¿Parece que se verifican los supuestos?
9. Realice la prueba de Shapiro para los residuales y comente el resultado.
10. Realice la prueba de Brausch-Pagan para los residuales y comente el resultado.
11. Utiliza la recta de regresión para interpolar dos valores y extrapolar uno. Comenta estos resultados.
12. Realice una tabla ANOVA e interprete el resultado.

```
import pandas as pd

df = pd.DataFrame({
    "numero_interrupciones_al_trabajador": [5,10,10,15,15,20,20,25],
    "calificacion_del_trabajador_en_la_prueba_de_hostilidad": [58,41,45,27,26,12,16,3]
})

df
```

	numero_interrupciones_al_trabajador	calificacion_del_trabajador_en_la_prueba_de_hostilidad
0	5	58
1	10	41
2	10	45
3	15	27
4	15	26
5	20	12
6	20	16
7	25	3

Próximos pasos: Generar código con `df` New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 2 columns):
 #   Column                                                  Non-Null Count  Dtype
---  ------                                                  --------------  -----
```

```
 0   numero_interrupciones_al_trabajador                     8 non-null      int64
 1   calificacion_del_trabajador_en_la_prueba_de_hostilidad  8 non-null      int64
dtypes: int64(2)
memory usage: 260.0 bytes
```

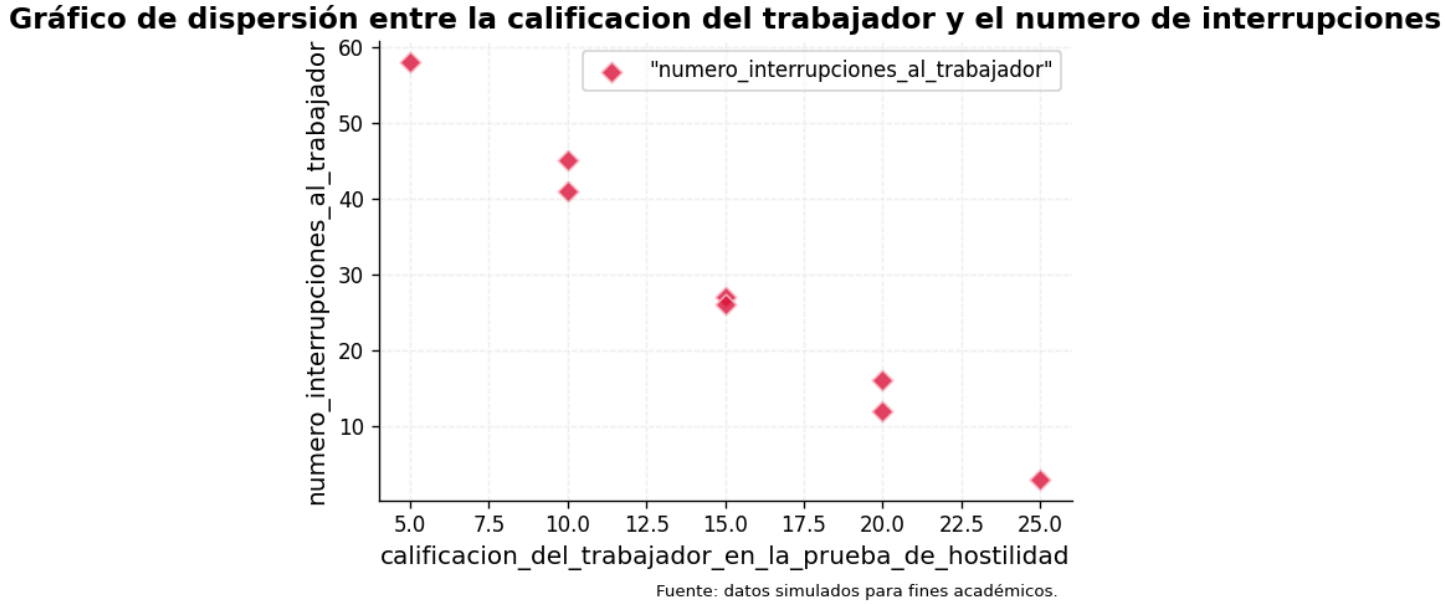
```
df.describe()
```

	numero_interrupciones_al_trabajador	calificacion_del_trabajador_en_la_prueba_de_hostilidad
count	8.000000	8.000000
mean	15.000000	28.500000
std	6.546537	18.462317
min	5.000000	3.000000
25%	10.000000	15.000000
50%	15.000000	26.500000
75%	20.000000	42.000000
max	25.000000	58.000000

```
# 1. Establesca una variable dependiente ( Y )
# y una variable independiente ( X ).

x = df["numero_interrupciones_al_trabajador"]             # variable independiente (con la que se cuenta)
y = df["calificacion_del_trabajador_en_la_prueba_de_hostilidad"]        # variable de interés (variable dependiente)
```

```
# @title
# 2. Realice un diagrama de dispersión para estos datos.
import matplotlib.pyplot as plt

# --- Configuración general del gráfico ---
plt.figure(
    figsize=(6, 4),   # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120           # resolución del gráfico
)

# --- Gráfico de dispersión ---
plt.scatter(
    x, y,
    marker="D",       # forma
    color='crimson',     # color de los puntos
    edgecolor='pink',    # borde de los puntos
    alpha=0.8,            # transparencia
    s=50,                 # tamaño de los puntos
    label='"numero_interrupciones_al_trabajador"' # etiqueta para la leyenda
)

# --- Gráfico de línea ---
# plt.plot(
#     x, y,
#     color='orange',   # color de la línea
#     linewidth=2.2,        # grosor de la línea
#     linestyle='-',        # estilo de línea
#     marker='o',           # marcador en cada punto
#     markersize=5,         # tamaño del marcador
#     markerfacecolor='white',
#     markeredgecolor='black',
#     label='Datos observados'
# )

# --- Título ---
plt.title(
    'Gráfico de dispersión entre la calificacion del trabajador y el numero de interrupciones',
    fontsize=14,
    fontweight='bold'
)

# --- Etiquetas de los ejes ---
plt.xlabel(
    'calificacion_del_trabajador_en_la_prueba_de_hostilidad',
    fontsize=12
)

plt.ylabel(
    'numero_interrupciones_al_trabajador',
    fontsize=12
```

```
)

# --- Fuente de los ticks ---
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# --- Márgenes ---
plt.margins(x=0.05, y=0.05)  # espacio extra alrededor de los datos
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Para eliminar márgenes completamente, usar:
# plt.margins(0)

# --- Cuadrícula (opcional, pero didáctica) ---
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.1,
    color="gray"
)

# --- Leyenda ---
plt.legend(
    fontsize=10,
    loc='best',
    frameon=True
)

# --- Nota al pie ---
plt.text(
    0.4, -0.2,
    'Fuente: datos simulados para fines académicos.',
    fontsize=8,
    ha='left',
    va='center',
    transform=plt.gca().transAxes
)

# --- Guardar gráfico ---
plt.savefig(
    "grafico de dispersión",
    bbox_inches='tight'
    )

plt.show()
```




```
#4.Calcule el coeficiente de correlación e interprete el resultado.
from scipy.stats import pearsonr
```

```
# Test de Pearson
# H0: rho = 0     (No hay correlación)
# H1: rho ≠ 0     (Sí hay correlación)
# alpha = 0.05

r, valor_p = pearsonr(x, y)

print(f"Coeficiente de correlación: {r: 0.4f}")
print(f'Valor p: {valor_p: 0.4f}')
```

```
Coeficiente de correlación: -0.9928
Valor p:  0.0000
```

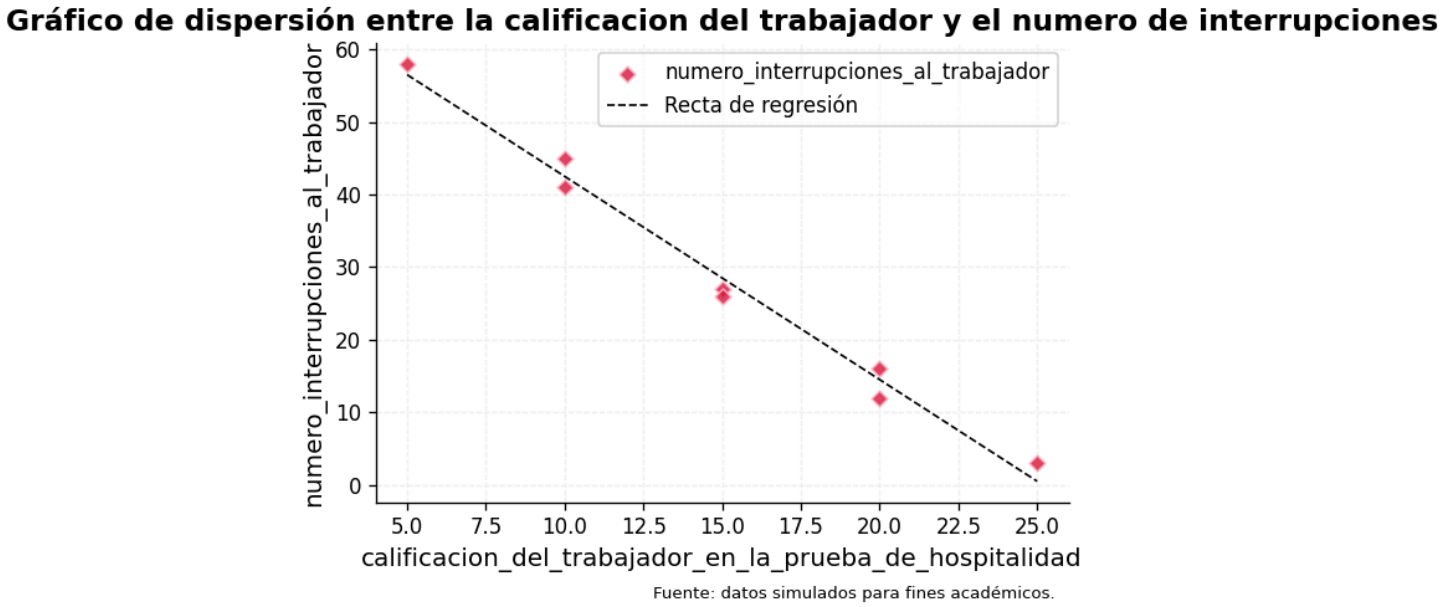
```
# 6. Obtenga la recta de regresión ajustada y grafíquelo sobre el
# gráfico de dispersión

import statsmodels.api as sm
x_constante = sm.add_constant(x)
modelo = sm.OLS(y, x_constante).fit()
y_calculada = modelo.predict(x_constante)

modelo.params
```

	0
const	70.5
numero_interrupciones_al_trabajador	-2.8

dtype: float64

```
# @title
# 2. Realice un diagrama de dispersión para estos datos.
import matplotlib.pyplot as plt

# --- Configuración general del gráfico ---
plt.figure(
    figsize=(6, 4),   # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120           # resolución del gráfico
)

# --- Gráfico de dispersión ---
plt.scatter(
    x, y,
    marker="D",       # forma
    color='crimson',     # color de los puntos
    edgecolor='pink',    # borde de los puntos
    alpha=0.8,            # transparencia
    s=30,                 # tamaño de los puntos
    label='numero_interrupciones_al_trabajador' # etiqueta para la leyenda
)

# --- Gráfico de línea ---
plt.plot(
    x, y_calculada,
    color='black',   # color de la línea
    linewidth=1.0,        # grosor de la línea
    linestyle='--',        # estilo de línea
    marker='o',           # marcador en cada punto
    markersize=0,         # tamaño del marcador
    markerfacecolor='white',
    markeredgecolor='black',
    label='Recta de regresión'
)

# --- Título ---
plt.title( 'Gráfico de dispersión entre la calificacion del trabajador y el numero de interrupciones',
    fontsize=14,
    fontweight='bold'
)

# --- Etiquetas de los ejes ---
plt.xlabel(
    'calificacion_del_trabajador_en_la_prueba_de_hospitalidad',
    fontsize=12
)

plt.ylabel(
    'numero_interrupciones_al_trabajador',
```

```
    fontsize=12
)

# --- Fuente de los ticks ---
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# --- Márgenes ---
plt.margins(x=0.05, y=0.05)  # espacio extra alrededor de los datos
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Para eliminar márgenes completamente, usar:
# plt.margins(0)

# --- Cuadrícula (opcional, pero didáctica) ---
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.1,
    color="gray"
)

# --- Leyenda ---
plt.legend(
    fontsize=10,
    loc='best',
    frameon=True
)

# --- Nota al pie ---
plt.text(
    0.4, -0.2,
    'Fuente: datos simulados para fines académicos.',
    fontsize=8,
    ha='left',
    va='center',
    transform=plt.gca().transAxes
)

# --- Guardar gráfico ---
plt.savefig(
    "grafico de dispersión",
    bbox_inches='tight'
    )

plt.show()
```




```
# 5. Calcule el coeficiente de determinación e interprete el resultado.
from sklearn.metrics import r2_score  # recomendada
```

```
r2 = r2_score(y, y_calculada)
print(f'Coeficiente de determinación: {r2: 0.2%}\n')
```

```
Coeficiente de determinación:  98.58%
```

```
modelo.params
```

	0
const	70.5
numero_interrupciones_al_trabajador	-2.8

dtype: float64

```
# 7. Obtenga un intervalo de confianza del 95% para la pendiente de la recta de
# regresión ajustada ( b1 )

# intervalo de confianza = 1 - alpha

modelo.conf_int(alpha = 0.05)
```

	0	1
const	65.051384	75.948616
numero_interrupciones_al_trabajador	-3.136296	-2.463704

```
# 8. Calcule los residuales y trace un nuevo gráfico de dispersión. Comente,
# ¿Parece que se verifican los supuestos?

# Los residuales son la diferencia entre el valor real y el valor que da el
# modelo

# residuales = y - y_calculada
residuales = modelo.resid

residuales
```

	0
0	1.5
1	-1.5
2	2.5
3	-1.5
4	-2.5
5	-2.5
6	1.5
7	2.5

dtype: float64

```
# @title
# 2. Realice un diagrama de dispersión para estos datos.
import matplotlib.pyplot as plt

# --- Configuración general del gráfico ---
plt.figure(
    figsize=(6, 4),   # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120           # resolución del gráfico
)

# --- Gráfico de dispersión ---
plt.scatter(
    x, residuales,
    marker="D",       # forma
    color='crimson',     # color de los puntos
    edgecolor='pink',    # borde de los puntos
    alpha=0.9,            # transparencia
    s=30,                 # tamaño de los puntos
    label='numero_interrupciones_al_trabajador' # etiqueta para la leyenda
)
```

```
plt.axhline(
    y=0,     # Donde está la línea horizontal
    label="Recta de regresión", # Etiqueta
    linestyle = "--", # Estilo de línea
    color = "black",  # color
    linewidth=1.0,   # ancho de línea
    )

# --- Gráfico de línea ---
# plt.plot(
#     x, y_calculada,
#     color='black',   # color de la línea
#     linewidth=1.0,        # grosor de la línea
#     linestyle='--',        # estilo de línea
#     marker='o',           # marcador en cada punto
#     markersize=0,         # tamaño del marcador
#     markerfacecolor='white',
#     markeredgecolor='black',
#     label='Recta de regresión'
# )

# --- Título ---
plt.title(
    'Gráfico de dispersión entre la calificacion del trabajador y el numero de interrupciones',
    fontsize=14,
    fontweight='bold'
)

# --- Etiquetas de los ejes ---
plt.xlabel(
    'calificacion_del_trabajador_en_la_prueba_de_hospitalidad',
    fontsize=12
)

plt.ylabel(
    'numero_intterrupciones_al_trabajador',
    fontsize=12
)
```